POLITECHNIKA
BYDGOSKA
Wydział Telekomunikacji,
Informatyki i Elektrotechniki

ENGINEER'S THESIS
field of study:
Information and Communication Technology

Optymalizacja kosztów usług chmurowych przy użyciu programowania liniowego oraz MS Power BI

Cost Optimization of Cloud Services using Linear Programming and MS Power BI

Kyrylo Brykov
119814

Supervisor: dr inż. Anna Witenberg

Bydgoszcz, February 2026

Metryka pracy dyplomowej

Dane ogólne

Nazwa Uczelni	Politechnika Bydgoska im. Jana i Jędrzeja Śniadeckich
Wydział	Telekomunikacji, Informatyki i Elektrotechniki
Kierunek	Teleinformatyka
Tryb studiów	stacjonarne
Dane autora	Kyrylo Brykov, 119814
Dane promotora	dr inż. Anna Witenberg

Dane dotyczące pracy dyplomowej

Język pracy	język angielski [EN]
Tytuł pracy	Optymalizacja kosztów usług chmurowych przy użyciu programowania liniowego oraz MS Power BI
Opis pracy	Celem pracy jest opracowanie deterministycznego modelu optymalizacyjnego dla alokacji zasobów w środowisku Multi-Cloud. Projekt implementuje silnik programowania liniowego oparty na języku Python w celu optymalizacji TCO przy uwzględnieniu złożonych ograniczeń oraz wizualizuje wpływ finansowy przy użyciu Power BI.
Typ pracy	inżynierska
Streszczenie	Praca adresuje problem nieefektywności alokacji zasobów w chmurze. Zaprojektowano autorski silnik optymalizacyjny oparty na Programowaniu Liniowym (biblioteka PuLP), który pozwala na znalezienie globalnego minimum TCO przy złożonych ograniczeniach budżetowych i technicznych. Badania wykazały przewagę podejścia algorytmicznego nad standardowymi metodami heurystycznymi. Wyniki zaprezentowano w raportach Power BI.
Słowa kluczowe	Chmura obliczeniowa, FinOps, Programowanie liniowe, Wielochmurowość, Optymalizacja kosztów, MS Power BI, Python, Heurystyki

Diploma thesis record

General information

University name	Bydgoszcz University of Science and Technology
Faculty	Telecommunications, Computer Science and Electrical Engineering
Field of study	Data Communications
Mode of study	full-time
Author's information	Kyrylo Brykov, 119814
Supervisor's information	dr inż. Anna Witenberg

Data regarding the diploma thesis

Language of thesis	English [EN]
Title of thesis	Cost Optimization of Cloud Services using Linear Programming and MS Power BI
Description	The objective is to develop a deterministic optimization model for Multi-Cloud resource allocation. The project implements a Python-based Linear Programming engine to optimize TCO against complex constraints and visualizes the financial impact using Power BI.
Type of thesis	Engineer's
Abstract	Cloud migration often leads to operational inefficiencies due to the complexity of multi-vendor pricing. This thesis proposes a mathematical framework using Linear Programming (LP) to solve the resource allocation problem. By comparing the LP model against a heuristic baseline using synthetic telemetry, the research demonstrates a measurable reduction in TCO and waste, visualized through Business Intelligence dashboards in Power BI.
Keywords	Cloud Computing, FinOps, Linear Programming, Multi-Cloud, Cost Optimization, MS Power BI, Python, Heuristics

Contents

Contents 4

1 Introduction and Research Problem Background 8

1.1 Cloud Computing Challenges: Cost Inflation and Inefficiency 8

1.2 The Emergence of FinOps: Engineering, Finance, and Business Alignment . . . 10

1.3 Identification of the Research Gap in Cloud Optimization Tools 11

1.4 Thesis Goal, Scope, and Research Justification 12

2 Theoretical Foundations and Optimization Methodologies 14

2.1 The FinOps Framework: Context and Challenges 14

2.2 Mathematical Fundamentals of Linear Programming 15

2.3 Heuristic vs. Algorithmic Approaches 17

2.4 Analytical Environment: Python, PuLP, and Power BI 18

3 Analytical Methodology and System Implementation 20

3.1 Data Schema and Telemetry Synthesis 20

3.2 Mathematical Formulation of the Optimization Logic 21

3.3 Implementation of the Optimal Solution Engine in Python (PuLP/SciPy) . . . 22

3.4 System Integration and Performance Metrics 23

3.4.1 Computational Efficiency Analysis 26

3.5 Comparison with Greedy Heuristic Baselines 26

3.5.1 Baseline Implementation Logic 29

3.5.2 Algorithmic Complexity Analysis 29

3.5.3 Sub-Optimal Convergence and Local Optima 30

3.6 Technical Implementation: Algorithmic Test Matrix 30

3.6.1 Domain 1: Deterministic Financial & Resource Vectors (Scenarios 1, 2, 4, 6) . . . 31
3.6.2 Domain 2: Dynamic Risk & Performance Governance (Scenarios 3, 7, 9, 12) . . . 32
3.6.3 Domain 3: Continuous Storage & Lifecycle Dynamics (Scenarios 5, 10, 13) . . . 32
3.6.4 Domain 4: Multi-Cloud Policy & Global Convergence (Scenarios 8, 11, 14, 15) . . . 32

4 Results Analysis and Business Intelligence Visualization 34
4.1 Experimental Context: The Controlled Environment . . . 34
4.2 Experimental Results: Level 1 - Isolated Optimization . . . 35
4.2.1 Group 1: Deterministic Resource Allocation . . . 35
4.2.2 Group 2: Handling Risk and Multiple Constraints . . . 37
4.3 Experimental Results Level 2 - Advanced Multi-Constraint Optimization . . . 43
4.3.1 Group 1: Financial & Capital Constraints (Scenarios 6 & 9) . . . 43
4.3.2 Group 2: Governance & Policy Constraints (Scenarios 7 & 8) . . . 45
4.3.3 Group 3: Storage Optimization (Scenario 10) . . . 47
4.4 Experimental Results Level 3 - Global & Multi-Objective Optimization . . . 47
4.4.1 Scenario 11: Budget Constrained Multi-Cloud Optimization . . . 48
4.4.2 Quantifying the "Tax" of Compliance (Scenarios 12 & 14) . . . 49
4.4.3 Scenario 13: Storage Migration & Lifecycle Arbitrage . . . 51
4.4.4 Group 4: Integrated Optimization (Scenario 15) . . . 53

5 Summary and Conclusions 56
5.1 Synthesis of Research Findings . . . 56
5.2 The Deterministic Advantage: Linear Programming vs. Heuristics . . . 57
5.3 Operational Trade-offs and Implementation Costs . . . 58
5.4 Final Conclusions . . . 59

6 Project Evolution and Future Development Paths 60
6.1 Evolution of the Thesis Concept . . . 60
6.2 Research Limitations and Data Acquisition Challenges . . . 60
6.3 Future Directions: From Script to Platform . . . 61
6.3.1 Transition to SaaS Architecture . . . 61
6.3.2 Integration of Predictive Machine Learning . . . 62

6.3.3 Stress-Testing in Non-Ideal Telemetry Environments 62
6.3.4 High-Dimensional Scalability and Latency Benchmarking 62

Bibliography **64**

List of Figures **67**

List of Tables **69**

Spis treści

1. Wstęp i tło problemu badawczego **8**
1.1. Wyzwania Cloud Computing: Inflacja kosztów i nieefektywność 8
1.2. Narodziny FinOps: Synergia inżynierii, finansów i biznesu 10
1.3. Identyfikacja luki badawczej w narzędziach optymalizacji chmury 11
1.4. Cel pracy, zakres i uzasadnienie badawcze 12

2. Podstawy teoretyczne i metodyki optymalizacji **14**
2.1. Framework FinOps: Kontekst i wyzwania 14
2.2. Matematyczne podstawy Programowania Liniowego (LP) 15
2.3. Podejście heurystyczne a algorytmiczne 17
2.4. Środowisko analityczne: Python, PuLP i Power BI 18

3. Metodyka analityczna i implementacja systemu **20**
3.1. Schemat danych i przetwarzanie wstępne telemetrii Multi-Cloud 20
3.2. Matematyczne sformułowanie logiki optymalizacyjnej 21
3.3. Implementacja silnika optymalizacji w Pythonie (PuLP/SciPy) 22
3.4. Integracja systemu i metryki wydajności 23
3.5. Porównanie z bazowymi algorytmami heurystycznymi 26
3.6. Implementacja techniczna: Macierz testowa algorytmów 30

4. Analiza wyników i wizualizacja Business Intelligence **34**
4.1. Kontekst eksperymentalny: Środowisko kontrolowane 34
4.2. Wyniki eksperymentalne: Poziom 1 – Izolowana optymalizacja 35
4.3. Wyniki eksperymentalne: Poziom 2 – Zaawansowana optymalizacja wielokryterialna 42
4.4. Wyniki eksperymentalne: Poziom 3 – Globalna optymalizacja wieloobiektowa 47

5. Podsumowanie i wnioski **55**
5.1. Synteza wyników badawczych 55
5.2. Przewaga deterministyczna: Programowanie Liniowe vs. Heurystyka 56
5.3. Kompromisy operacyjne i koszty wdrożenia 57
5.4. Wnioski końcowe 58

6. Ewolucja projektu i przyszłe ścieżki rozwoju **59**
6.1. Ewolucja koncepcji pracy 59
6.2. Ograniczenia badawcze i wyzwania związane z pozyskiwaniem danych 59
6.3. Kierunki rozwoju: Od skryptu do platformy SaaS 60

Bibliografia **63**
Spis rysunków **66**
Spis tabel **68**

Chapter 1

Introduction and Research Problem Background

1.1 Cloud Computing Challenges: Cost Inflation and Inefficiency

Enterprise infrastructure has shifted from static capital expenditure (CAPEX) to a dynamic operational expenditure (OPEX) model, driven by the "pay-as-you-go" billing structure of cloud computing. And while early literature by Armbrust et al. [3] positioned the cloud as a utility to reduce hardware burdens, modern implementations often suffer from stranded capacity, otherwise known as waste, and inefficiency. These inefficiencies frequently come from challenges in normalizing input data for analysis as the scale increases. As organizations pursue agility, the volume of cloud usage data often exceeds the limits of what a human can analyze, leading to cost inflation.

The technical complexity of this structure is significant. The National Institute of Standards and Technology (NIST) definition of cloud computing is characterized by "rapid elasticity" and "on-demand self-service" [17], which has led to a vast array of configuration options. As of late 2025, one of the cloud providers mentioned in this study, AWS, is now offering over 750 instance types, each with specific performance vectors [2]. Simultaneously tracking these dynamic variables, e.g., price volatility, Spot instances, and multi-year discounts, is increasingly difficult with various configuration options. This complexity creates discrepancies between expected and actual costs, a challenge noted by Buyya et al. [6] regarding resource commoditization. Optimization has become a multi-dimensional problem requiring informed technical decisions,

such as instance selection, which must align with both the financial constraint and the system resilience.

Figure 1.1: Comparison of AWS EC2 Pricing Models: On-Demand, Reserved, and Spot Instances

Current native tools, such as AWS Cost Explorer, often suffer from a lack of ability in future estimation, falling into the "greedy heuristic" trap. Where an algorithm buys an instance of computing power here and now, forgetting about the limitations that already existed, causing a big complication for the finance's budget constraints. These platforms typically use algorithms that optimize locally, such as selecting the best computing instance here and now, without considering the interdependent budget constraints. As Beloglazov et al. [4] note, heuristics are often insufficient for complex, enterprise-scale trade-offs where variables are coupled. Reliance on these methods can lead to an efficiency gap, where significant Total Cost of Ownership (TCO) savings are missed, as tools are simply unable to take into account massive multi-dimensional resources and various budget constraints.

And while Beloglazov et al. [4] argue that the dynamic resource allocation is extremely hard, stating that sub-optimal heuristics are essential, Chaisiri et al. [7] and Chen et al. [8] counter that cost-specific provisioning can be modeled deterministically. By utilizing Dantzig's [9] linear programming algorithms, this research prioritizes a global financial optimum rather than local heuristic approximations.

This technical imperfection causes friction between Engineering and Finance. Engineers often over-provision resources to ensure reliability, a practice viewed as "waste" by Finance. Conversely, cost optimization teams enforce rigid **budgetary limits** that may prevent strategic long-term cost optimization, a constraint viewed as illogical by Engineers. Without a unified mathematical framework, neither group achieves the Global Optimum required for effective FinOps.

This research aims to address this impasse by redefining the optimization task as a mathematical, deterministic calculation. By utilizing a **Simplex-based Linear Programming**

(LP) approach, the model navigates the maze of varying constraints to identify the vector of improvement, ensuring a mathematically supported Global Optimum rather than a sub-optimal local solution found by heuristic approximations.

1.2 The Emergence of FinOps: Engineering, Finance, and Business Alignment

Financial Operations (FinOps) has emerged as a practice, or perhaps more accurately a culture, to manage the intersection of cloud technology and business finance. This shift addresses the operational inefficiencies resulting from the complexity of modern cloud catalogs. The "Rapid Elasticity" defined by Mell & Grance [17] now requires governance mechanisms beyond manual oversight. As this study analyzed, FinOps seeks to align engineering's need for a resourceful cloud structure with finance's need for fiscal predictability.

The FinOps life-cycle, "Inform, Optimize, Operate", aims to align spending with business value [22].

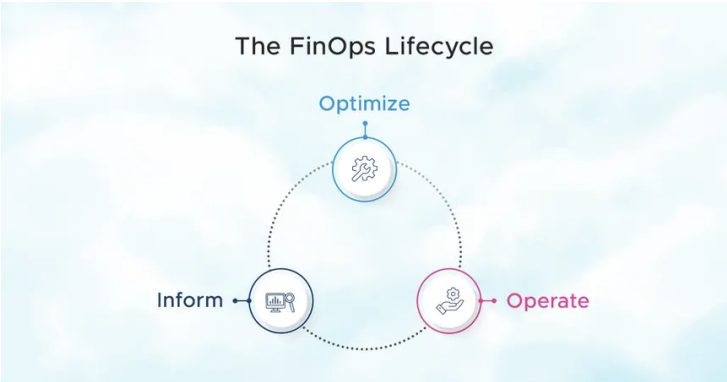


Figure 1.2: The FinOps Lifecycle: Inform, Optimize, and Operate

The "Inform" phase relies on input data to properly segregate and create billing reports. Without meticulous normalization of this data, the "Optimize" phase suffers from the limitations of standard heuristic approaches. This leads to recommendations based on historical averages, or simply what happened yesterday, rather than global TCO minimization. Consequently, the "Operate" phase inherits sub-optimal configurations. This results in continuous operational waste that inevitably creates friction with the finance department.

FinOps emphasizes "Cloud Unit Economics" to measure cost against business outcomes. However, implementing "shared ownership" is especially challenging because engineers face complex trade-offs. They usually have to choose between technical performance and **bud-**

getary constraints. Therefore, engineers often lean toward over-provisioning as a safety buffer while Finance imposes caps in an attempt to control the budget. This misunderstanding and lack of optimization usually lead to friction between the two groups.

This goal misalignment arises from the gap between engineering metrics (throughput, latency) and optimistic financial constraints (budget caps, unit costs). In the current world, these constraints are rarely realistic due to a lack of proper telemetry. Finance may view cloud costs as variable and unpredictable, while engineering views budget limits as hurdles they must overcome to reach **operational resilience**. Current tools often provide simple heuristics that ignore the marketing strategies employed by larger cloud providers, such as reserved instances, savings plans, and spot volatility. These tools lack the algorithmic depth to navigate the complicated maze of the solution space effectively.

To achieve "True Optimization Maturity," the industry relies on moving away from qualitative practices toward quantitative, mathematical, deterministic models. An "Optimal Solution Engine" is required to process the vast array of combinations and the complexity of cloud variables. By framing FinOps as a search for the Global Optimum within an LP framework, organizations are trying to align engineering performance with financial efficiency.

1.3 Identification of the Research Gap in Cloud Optimization Tools

The cloud cost management market frequently lacks mathematical depth in the standard tools that are offered. Organizations often heavily rely on native tools, such as AWS Cost Explorer, which are strictly limited by "Provider Silos" and reactive heuristics [4]. These tools, as the study discovered, typically operate within a closed loop, offering only localized suggestions based on what has already happened, and yet they do not account for multi-cloud environments or more complex ecosystems. This **heuristic limitation** is standing in the way of organizations trying to achieve the global minimum of their TCO.

A good example of such limitations can be noticed in commercial platforms like CloudHealth, which often function as black-box recommendation engines. The problem encountered is the lack of mathematical transparency, or evidence, which prevents stakeholders from verifying trade-offs. These recommendations to migrate workloads often lack context regarding **financial constraints** or technical vectors like IOPS density. This leads to **operational uncertainty**, where the engineers lean towards other solutions rather than the offered recommendations, hoping to avoid performance degradation [22].

Other existing tools, such as automated commitment management tools, are also struggling with multi-dimensional constraints. "Greedy" algorithms may be able to secure a discount for a compute-heavy instance, yet they do not consider storage or memory bottlenecks, leading to more budget ruin rather than optimization. In contrast, the chosen mode is based on a deterministic, mathematically supported **Linear Programming (LP)** model, which treats every resource required for system functionality as a variable in a system of linear inequalities. This approach enables the solver to navigate the maze of complex instance types and pricing models, aiming to find the most optimal solution [9]. This capability is essential in multi-cloud environments, where intelligent decision-making is required to manage diverse resources [29].

The gap is so noticeable between the tool capabilities and enterprise needs, that this study aims to define the efficiency gap that appears in cloud spend. Commercial tools effectively identify idle resources, yet are unable to always provide the best solution for the complex problem of finding the Global Optimum in a multi-cloud environment, which is a subject to a variety simultaneous constraints. They often lack the ability to optimize trade-offs between Spot instance risk versus pay-off, and On-Demand reliability under hard CAPEX ceilings.

This research sets a goal of providing a clearer mathematical analysis of the cloud compute costs and the possibility of presenting another solution as an alternative to current tools. By implementing a PuLP-based LP engine, the author aims to bridge the efficiency gap [19]. The proposed model provides mathematical transparency, connecting the technical demands of engineers with the objective function to minimize cost. This transition to a mathematically supported framework is necessary to manage the increasing complexity of cloud economics.

1.4 Thesis Goal, Scope, and Research Justification

This research aims to address the lack of mathematical precision in cloud resource allocation by developing a third-party cost optimization model using **Linear Programming**. Implemented via the Python-based PuLP library, this engine is designed to locate the **Global Optimum** of TCO across multi-dimensional multi-cloud environments. By framing the problem as a search within this multi-dimensional area, the model ensures that the fiscal efficiency can be reached without compromising technical performance, moving beyond the **limitations of greedy heuristics**.

The scope focuses on AWS and Microsoft Azure, and, at the same time, is aiming to address multi-cloud complexity. While the goal is to restrict technical boundaries to compute and storage resources, it accounts for variables such as vCPU-to-RAM ratios, IOPS throughput, and pricing tiers. **Microsoft Power BI** is integrated for the best showcase of a difference

an informed decision can make, as well as to visualize the structured output generated for this mathematically supported action that bridges the gap between system reliability and fiscal predictability.

The methodology used is justified by the significant **waste** in enterprise infrastructure. Basic analysis of the problem showcases simple, yet effective cost reduction possibilities through deterministic storage tiering. A result traditional heuristics often miss due to their inability to process long-term lifecycle costs [2]. The model will be validated through 15 experimental, synthesized scenarios, ranging from isolated rate optimization to integrated global governance, to demonstrate the ability to maintain a global minimum across diverse constraints, whilst focusing on showcasing the limitations such an approach may experience [22].

From an Information, Technology and Communication (ITC) perspective, this research synthesizes network resource management with business intelligence. The research aims to address the need for a technically robust and economically sustainable system by treating cloud expenditure as a variable in a network optimization problem. This approach satisfies the mathematical standards this thesis is aiming for while ensuring the operational resilience required by modern IT architectures.

Chapter 2

Theoretical Foundations and Optimization Methodologies

2.1 The FinOps Framework: Context and Challenges

The transition from CAPEX to OPEX has introduced a challenge for traditional IT departments that they are failing to handle. While the usual definition of cloud computing relies heavily on its elasticity, which often remains unused, [17], the economic reality often creates contradictions. The "pay-as-you-go" efficiency promised by early literature such as Armbrust et al. [3] may not always materialize. In practice, unmanaged elasticity, otherwise known as waste, creates budget issues and complications. The FinOps framework, as codified by Storment & Fuller [22], is setting a very high goal of creating a solution for the existing problem, going so far as creating the Inform, Optimize, Operate phases, yet it frequently fails at phase one due to the poor quality of data input, which this research refers to, as telemetry.

The encountered problem is that the "Inform" phase is rarely a straightforward reporting task, as it heavily relies on processing data to correctly identify important information. Raw telemetry, such as the AWS Cost and Usage Report (CUR), comes as a fragmented stream of billions of inconsistent records. Without normalization for the obtained data, these datasets are unusable and cannot be used for optimization. VanderLans [23] correctly identifies that analytical validity is often limited by input quality, implying that any optimization model built on unprocessed telemetry can accumulate "reporting debt." Furthermore, following standards such as the AWS Well-Architected Framework [1] requires specific cost attribution that heuristic tools often lack. Commercial tools like AWS Cost Explorer or Azure Advisor attempt to bypass this complexity by implementing rule-based heuristics, suggesting, that is, isolating

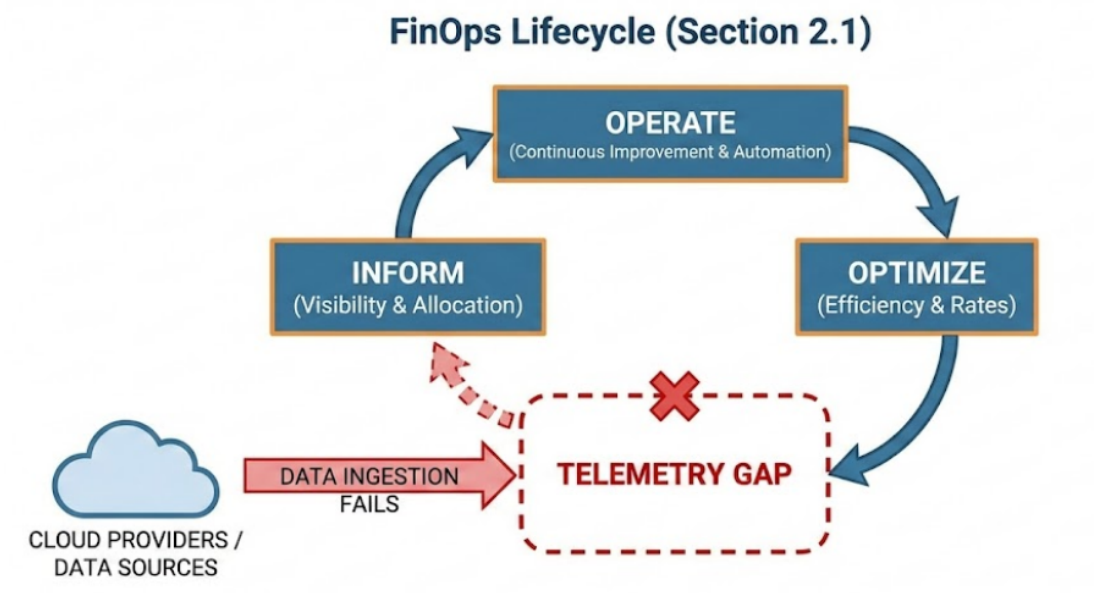


Figure 2.1: **The FinOps Lifecycle:** The crucial phases of FinOps, which are Inform, Optimize, Operate

instance downsizes based on simple CPU averages, yet they still fail to account for burst workloads or multi-cloud dependencies. Proper optimization cannot be a series of disconnected recommendations that are not supported by any evidence. This approach must be framed as a global mathematical problem solving for minimum cost under specific technical constraints, while including fiscal responsibilities.

This is where the engineering-finance tension peaks. Engineers, to whom reliability is the most important, naturally drift toward **waste**, over-provisioning as a safety buffer. Finance, meanwhile, demands a strict path with budget constraints in mind. As Deochake [11] argues in a comprehensive review of cloud cost models, FinOps is attempting to implement a solution where both parties are in agreement while following the optimization goal that was assigned. The conducted study aims to show that Linear Programming provides the necessary bridge, combining the engineers' goal for system stability with adherence to budget constraints set by finance.

2.2 Mathematical Fundamentals of Linear Programming

To provide control of multi-cloud allocation is a challenge highlighted by Zhang et al. [29] in the context of intelligent cloud operations. The world has to move beyond simple accounting

to a more deterministic approach, such as Linear Programming. The enterprise has to treat the cloud environment as not just a catalog of services, but as a complex multi-dimensional space. Every dimension represents a constrained resource, be it vCPU, RAM, or throughput, and the possible area that represents the narrow intersection where both technical performance and budgetary limits are satisfied.

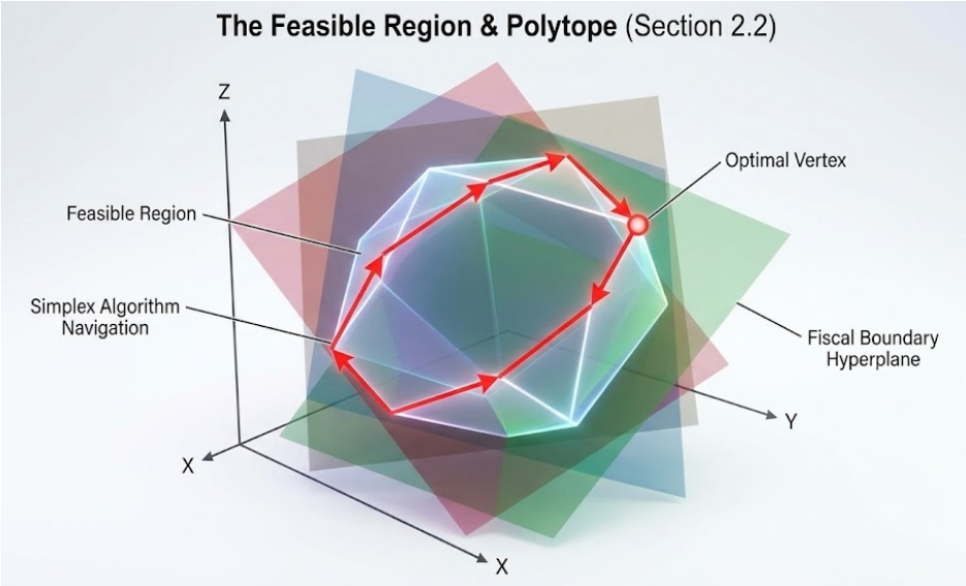


Figure 2.2: **Possible Solution Area Representation:** The solution space that is limited by technical constraints (blue) and budgetary limits (red). The Simplex algorithm is going through all the vertices and vectors in a search for the Global Optimum

This research defines the total cost Z as the summation of singular unit costs for selected Stock Keeping Units (SKUs), creating an objective function to be minimized:

$$\min Z = \mathbf{c}^T \mathbf{x} \tag{2.1}$$

This minimization is strictly limited by the intersection of business and technical demands:

$$\mathbf{A}\mathbf{x} \leq \mathbf{b} \tag{2.2}$$

$$\mathbf{x} \geq 0 \tag{2.3}$$

Here, the vector $\mathbf{x}$ represents a decision variable (allocation), while the matrix $\mathbf{A}$ codifies the constraints [9]. Most importantly, the waste variables in this system are not abstract; they quantify specific inefficiencies, **non-Optimized Elasticity**, that generate no value. While Chaisiri et al. [7] have looked into optimal cloud resource provisioning more in a theoretical context, this research aims to check the LP approach by applying it to synthesized, normalized

telemetry. This approach eliminates the noise inherent in raw billing logs, ensuring experimental control, while maintaining economic realism through the integration of actual provider pricing vectors. By explicitly defining the constraints, such as a mandatory minimum for IOPS, or a hard cap on OPEX, this study forces the model to take into account both sides of the coin, optimizing the configurations to offer a resilient solution within defined budgetary constraints.

2.3 Heuristic vs. Algorithmic Approaches

The current industry standard for cost optimization relies heavily on heuristics. This is primarily due to its benefits as a fast way to try and optimize the budget. However, while focusing on the "here and now," the tools often used for quick optimization are missing the real global optimum which is just a few steps ahead, as shown in Figure 2.3. Therefore, the greediness of current algorithms comes from prioritizing speed and fast savings, selecting fast and cheap instances, relying on reacting to yesterday's world with today's solutions, and trying to satisfy the immediate requirement rather than looking at the whole picture. This approach leads to only a local optimum, a cost reduction in computing that may unexpectedly spike storage transfer fees that the algorithm did not have enough depth to predict, or violate compliance requirements.

Beloglazov et al. [4] noted the limitations of such interdependent constraint handling. Yet also pointed out the difficulty in a deterministic approach. In the research, they argue that while in a multi-cloud complex structure, the heuristics may struggle, the difficulty of formulating clear constraints for a more deterministic solution is excessively difficult. In a complex multi-cloud architecture, heuristics are essential, they argue, yet in the modern era, this approach is fixing one inefficiency while creating another. As demonstrated by Chen et al. [8], algorithmic approaches provide far better competitive ratios compared to sub-optimal heuristics when attempting to manage online resource allocation. Their model is able to guarantee a global minimum for TCO, simply because they process the entire solution space rather than localized segments.

The strategy applied in this model is treating the limitations as a unified structure; the LP solver evaluates all constraints $\mathbf{A}$ simultaneously against the cost $\mathbf{c}$. Hence, the output becomes deterministic: as the specific configuration $\mathbf{x}$ that minimizes Z is not a suggestion, it is the mathematical floor of expenditure given the current constraints. This transparency is very important. If the solver returns "unreachable," it forces an informed, data-driven conversation about whether the budget is unrealistic or the technical requirements are over-providing. This transparency is eliminating the "black box" problem, which appears in heuristic algorithms.

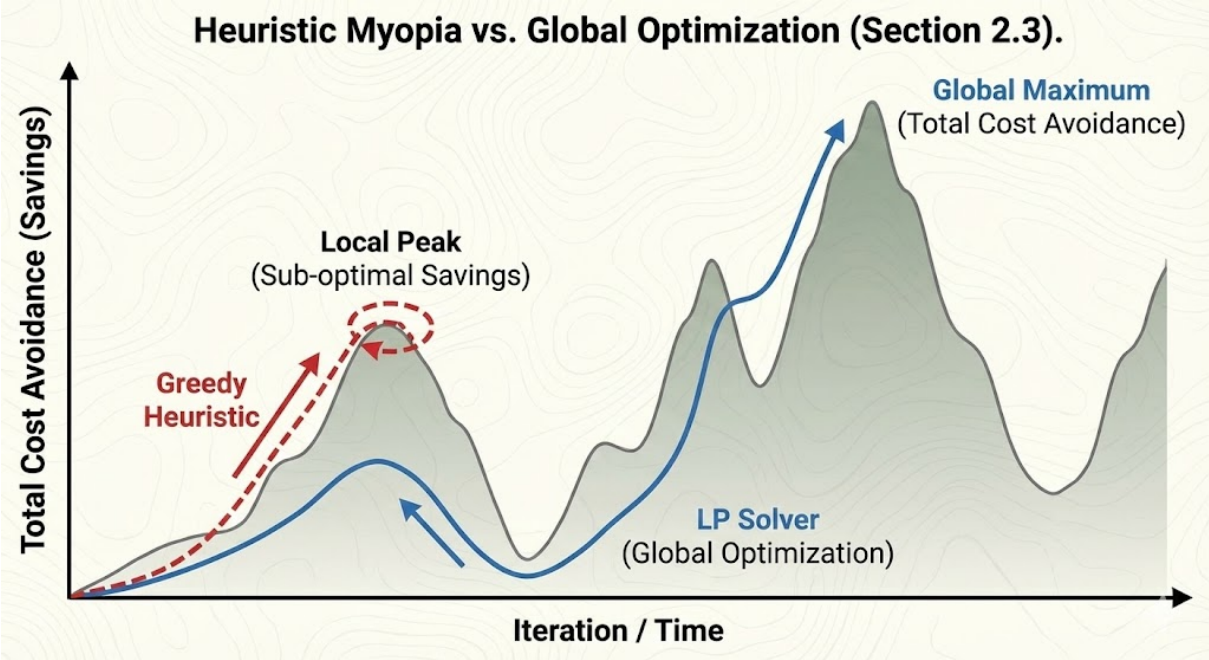


Figure 2.3: **Local vs. Global Optimization:** Heuristic methods often settle for a local optimum (A), while LP solvers evaluate the full constraint set to identify the global optimum (B)

2.4 Analytical Environment: Python, PuLP, and Power BI

The implementation of the model requires a computational engine capable of ingesting telemetry data and performing complex matrix operations. Standard spreadsheets were deemed insufficient for this task. This research utilized Python as the core orchestration layer to implement the Linear Programming model defined by Dantzig [9].

To ensure the mathematical validity of the engine, the study prioritizes experimental control over raw data variability. This approach enabled the generation of specific, isolated demand vectors—free from the noise and anomalies typical of raw production logs—to strictly test the solver's logic against defined constraints. The `PuLP` library was then used to map these controlled inputs into the optimization structure, ensuring that any identified efficiency gaps were the result of algorithmic superiority rather than data inconsistency.

The optimization logic is constructed using the `PuLP` library. This tool functions as a critical abstraction layer, enabling the definition of objective functions and constraints within a Pythonic environment while leveraging the power of underlying high-performance solvers [19]. This architecture establishes a clear separation between the modeling language and the

solver implementation, facilitating a transparent framework where the multi-level constraints of these experimental scenarios could be dynamically adjusted.

However, the raw numerical output generated by the optimization engine provides limited immediate value to business stakeholders. To bridge the gap between mathematical proof and informed decision-making, Microsoft Power BI was integrated as the visualization layer. Utilizing Data Analysis Expressions (DAX) patterns [12], a model is created containing the quantitative delta between the heuristic baseline and the LP-optimized state. This process transforms abstract computational savings into clear, actionable intelligence [21], validating that the recommended "global optimum" is not merely theoretically sound, but operationally viable.

Chapter 3

Analytical Methodology and System Implementation

This chapter describes the engineering methodology used to construct the cost optimization engine. The research begins by defining the data generation strategy. It explains why synthesized normalized telemetry was chosen for testing instead of raw data preprocessing. Next, the chapter details the mathematical formulation of the objective function and constraints within the Linear Programming framework. Finally, the system's software architecture is described. This section focuses on the role of the `PuLP` library and the test matrix used to benchmark the system against standard heuristic baselines.

3.1 Data Schema and Telemetry Synthesis

A major engineering challenge in validating this engine was establishing a reliable data baseline. In production environments, billing exports like the AWS Cost and Usage Report (CUR) are unstructured. They also frequently contain tagging errors. To isolate the solver's efficiency from this data noise, the research uses a **Synthetic Telemetry Generation** strategy [23].

Data Origin and Synthesis: Experimental control was prioritized over using variable production logs. Early tests with raw billing data showed that inconsistent formatting caused the solver to fail. Therefore, while the data schema mirrors standard CUR and ACUD files, the usage telemetry itself is synthetic.

Python scripts were used to generate dynamic demand patterns. These scripts programmed specific volatility profiles to mimic enterprise load fluctuations. However, to guarantee eco-

nomic reality, all financial limits come directly from the AWS Price List API and Azure Retail Prices API (2024). This hybrid approach tests the system against complex demand scenarios but avoids the privacy restrictions of real corporate data.

Each telemetry record is converted into a feature vector $\mathbf{r}_i$. This vector contains essential metrics: vCPU count, memory (GiB), and IOPS. At this stage, the unit cost is linked to the objective function. This phase also handles dimensionality reduction [3]. Detailed usage data is grouped into fixed time blocks (hourly or daily). This reduces the computational workload for the solver. It forces the engine to focus on structural cost patterns rather than transient noise. This optimizes the feasible region without losing data fidelity.

Standard industry tools (e.g., AWS Cost Explorer) often average metrics over long windows, such as 24 hours, which smooths out utilization spikes. Unfortunately, this undermines the actual workload capacity calculation. By averaging out these bursts, tools often recommend "right-sizing" configurations that lack the headroom required for peak load. This introduces stability risks [4]. To fix this, the proposed pipeline preserves high-resolution volatility patterns. The solver must optimize based on Peak Load requirements rather than averages. This ensures the recommended "Global Optimum" remains stable during stress periods.

3.2 Mathematical Formulation of the Optimization Logic

To translate operational needs into a format solvable by the Simplex algorithm, infrastructure rules were recreated as mathematical constraints. This frames the cost reduction problem as a search for a global optimum within a **multi-dimensional solution space**. This approach avoids the limitations of rule-based heuristics.

The study defines a set of decision variables $x_{i,j,k}$. These represent the quantity of a specific server SKU i required from provider j, using payment model k. This structured input ensures that all set conditions serve as active constraints. These conditions include financial limits, technical minimums, and risk policies.

The core of the system is the objective function. It was designed to minimize TCO, rather than focusing solely on OPEX like many commercial tools. Hence, the model integrates upfront CAPEX requirements to provide a full economic view. The objective function Z is the **calculated sum** of the cost vector and the allocation vector:

$$\min Z = \sum_{i \in I} \sum_{j \in J} \sum_{k \in K} (C_{i,j,k}^{hourly} \cdot T + C_{i,j,k}^{upfront}) \cdot x_{i,j,k} \tag{3.1}$$

This minimization is subject to hard constraints that define the feasible solution region:

1. **Compute Capacity Constraint:** The grouped vCPU count must meet the minimum R_{vCPU}. This prevents performance degradation:

$$\sum_{i,j,k} vCPU_i \cdot x_{i,j,k} \geq R_{vCPU} \tag{3.2}$$

2. **Memory Allocation Constraint:** The total RAM must satisfy the workload's memory requirement R_{RAM}:

$$\sum_{i,j,k} RAM_i \cdot x_{i,j,k} \geq R_{RAM} \tag{3.3}$$

3. **Budget Constraint (CAPEX Limit):** Upfront spend is capped at budget B_{CAPEX}. This forces the solver to trade off lower long-term rates against immediate liquidity:

$$\sum_{i,j,k} C_{i,j,k}^{upfront} \cdot x_{i,j,k} \leq B_{CAPEX} \tag{3.4}$$

4. **Risk Management (Spot Limit):** To maintain availability, reliance on volatile Spot instances is capped at threshold V_{Spot}:

$$\sum_{i,j,Spot} vCPU_i \cdot x_{i,j,Spot} \leq V_{Spot} \tag{3.5}$$

These constraints operate simultaneously. A decision to increase Reserved Instances to lower Z might be blocked by the CAPEX constraint (3.4). This forces the solver to find an alternative vertex. This simultaneous resolution prevents the cyclical dependency problem often seen in heuristic optimization, where fixing one metric breaks another.

3.3 Implementation of the Optimal Solution Engine in Python (PuLP/SciPy)

This section details the system architecture used to execute the mathematical model. Python was selected for the optimization engine due to its robust ecosystem for matrix operations. It also provides easy interfaces with solvers via the `PuLP` library [19].

The architecture uses `PuLP` to handle the underlying solver mechanics. This allows the system to define the problem in high-level logic. It delegates the actual solution computation to optimized C libraries (such as CBC or GLPK) [25].

The study uses `LpVariable.dicts` to generate decision variables dynamically based on the input SKU catalog. This addresses the issue of variable scale. To align with physical provisioning realities, variables are strictly typed as `LpInteger` [7].

The objective function was constructed using vectorization. This minimizes iteration overhead. By calculating the 3-year amortized cost (incorporating `HOURS_IN_3_YEARS`) within the vector summation, the solver is forced to evaluate the long-term economic impact of every allocation decision.

Listing 3.1: Vectorized Objective Function for TCO Minimization

```
        model += lpSum([
        instances[i]['upfront'] * instance_vars[i] +
        instances[i]['cost'] * instance_vars[i] * HOURS_IN_3_YEARS
        for i in instances
        ])
```

This approach contrasts with the iterative loops found in basic scripts. It significantly reduces the model build time. The selected strategy ensures that the output is a mathematically verified global optimum for the given constraints, rather than an approximation.

3.4 System Integration and Performance Metrics

The technical architecture operates as a deterministic pipeline. It converts synthesized infrastructure telemetry into actionable financial intelligence through algorithmic processing.

Execution begins by defining variables within the Python environment. Rather than relying on post-processing grouping, the script maps the delta between the heuristic baseline and the optimized result directly to a Python dictionary. This ensures accurate calculations.

Following the calculation, the engine serializes the optimization results into CSV files. Since standard Microsoft Excel struggled with the complexity of the dataset, the design creates a specific, separate telemetry record assigned to a distinct scenario. This allows for a concise view of the optimization as a whole.

The repository serves as the integration point for the visualization layer. It acts as the handshake between the backend (Python) and the analytics frontend (Power BI).

Data input is handled via the Power BI interface. It establishes a read-only connection to the CSV files while allowing for additional data transformation if necessary. This approach maintains a clear separation between data processing and the visualization layer.

Strict schema validation is enforced during ingestion. The interface verifies column types, specifically ensuring percentage and currency floats are correctly typed. This prevents downstream reporting errors.

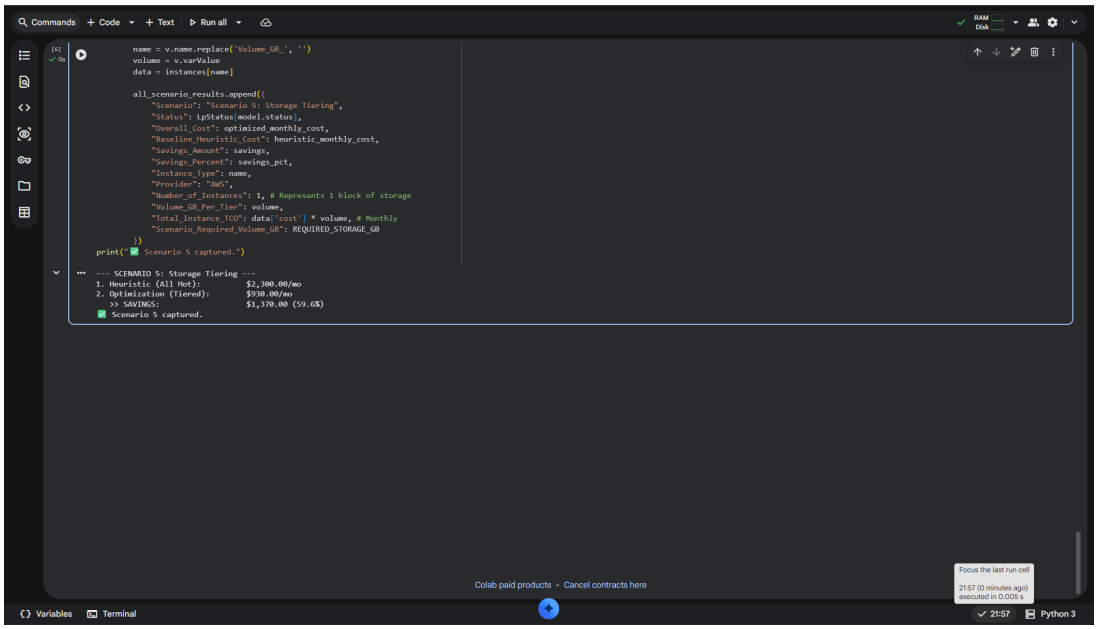
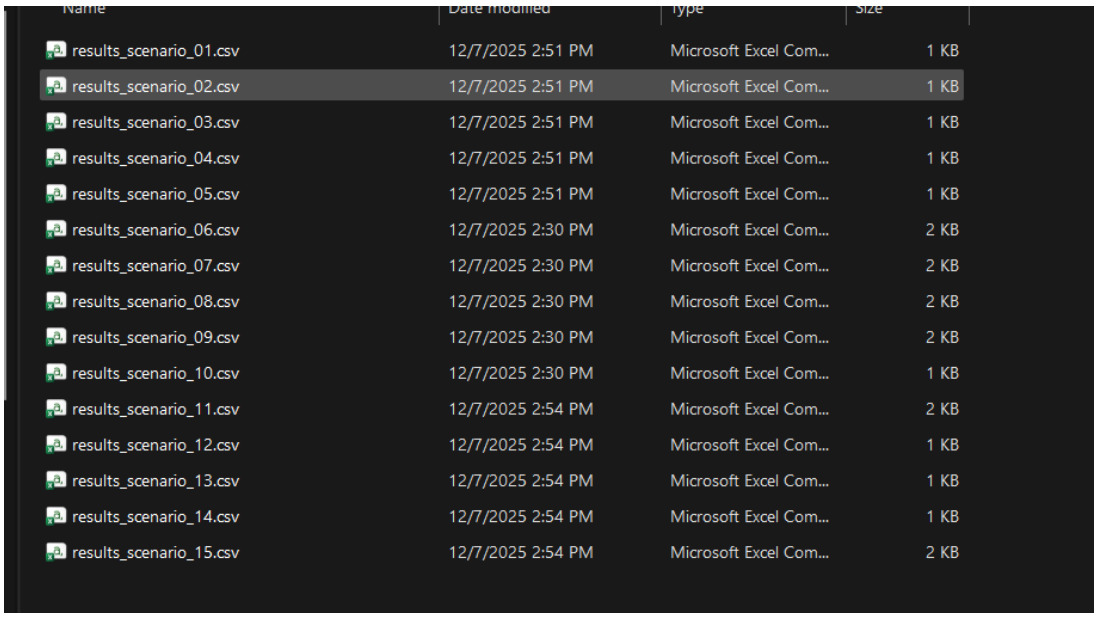


Figure 3.1: Cloud-Hosted Computational Environment (Google Colab) utilizing the IPython kernel



Name	Date modified	Type	Size
results_scenario_01.csv	12/7/2025 2:51 PM	Microsoft Excel Com...	1 KB
results_scenario_02.csv	12/7/2025 2:51 PM	Microsoft Excel Com...	1 KB
results_scenario_03.csv	12/7/2025 2:51 PM	Microsoft Excel Com...	1 KB
results_scenario_04.csv	12/7/2025 2:51 PM	Microsoft Excel Com...	1 KB
results_scenario_05.csv	12/7/2025 2:51 PM	Microsoft Excel Com...	1 KB
results_scenario_06.csv	12/7/2025 2:30 PM	Microsoft Excel Com...	2 KB
results_scenario_07.csv	12/7/2025 2:30 PM	Microsoft Excel Com...	2 KB
results_scenario_08.csv	12/7/2025 2:30 PM	Microsoft Excel Com...	2 KB
results_scenario_09.csv	12/7/2025 2:30 PM	Microsoft Excel Com...	2 KB
results_scenario_10.csv	12/7/2025 2:30 PM	Microsoft Excel Com...	1 KB
results_scenario_11.csv	12/7/2025 2:54 PM	Microsoft Excel Com...	2 KB
results_scenario_12.csv	12/7/2025 2:54 PM	Microsoft Excel Com...	1 KB
results_scenario_13.csv	12/7/2025 2:54 PM	Microsoft Excel Com...	1 KB
results_scenario_14.csv	12/7/2025 2:54 PM	Microsoft Excel Com...	1 KB
results_scenario_15.csv	12/7/2025 2:54 PM	Microsoft Excel Com...	2 KB

Figure 3.2: Serialization of optimization outputs into CSV files for visualization ingestion

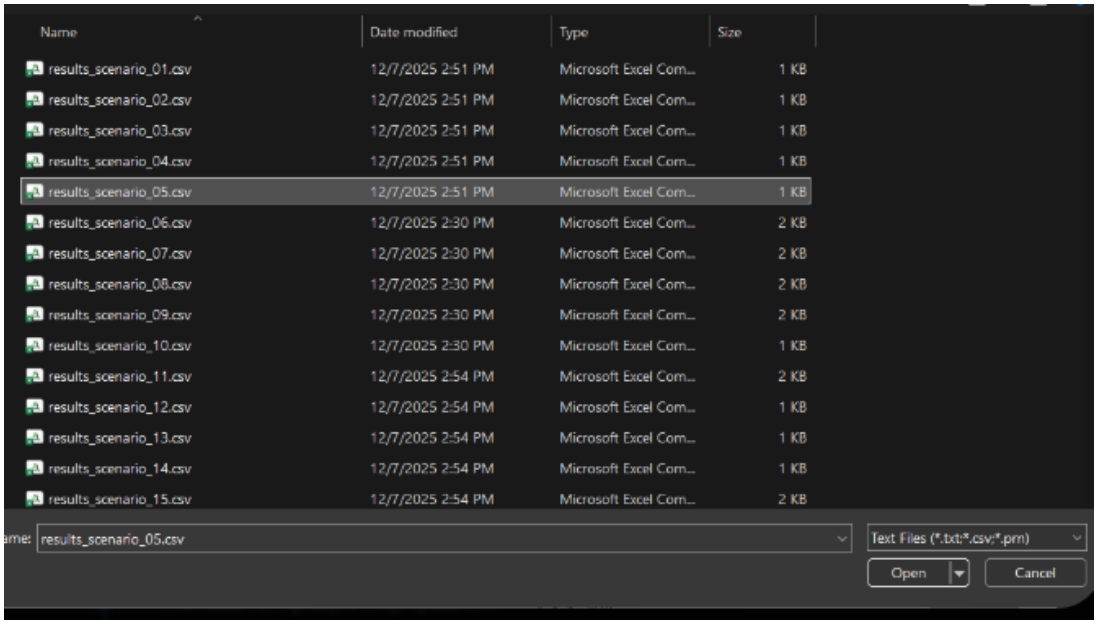


Figure 3.3: Selection of specific scenario files for import into the visualization layer

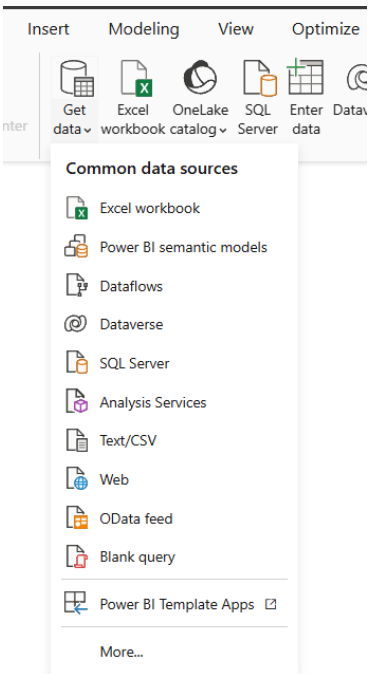
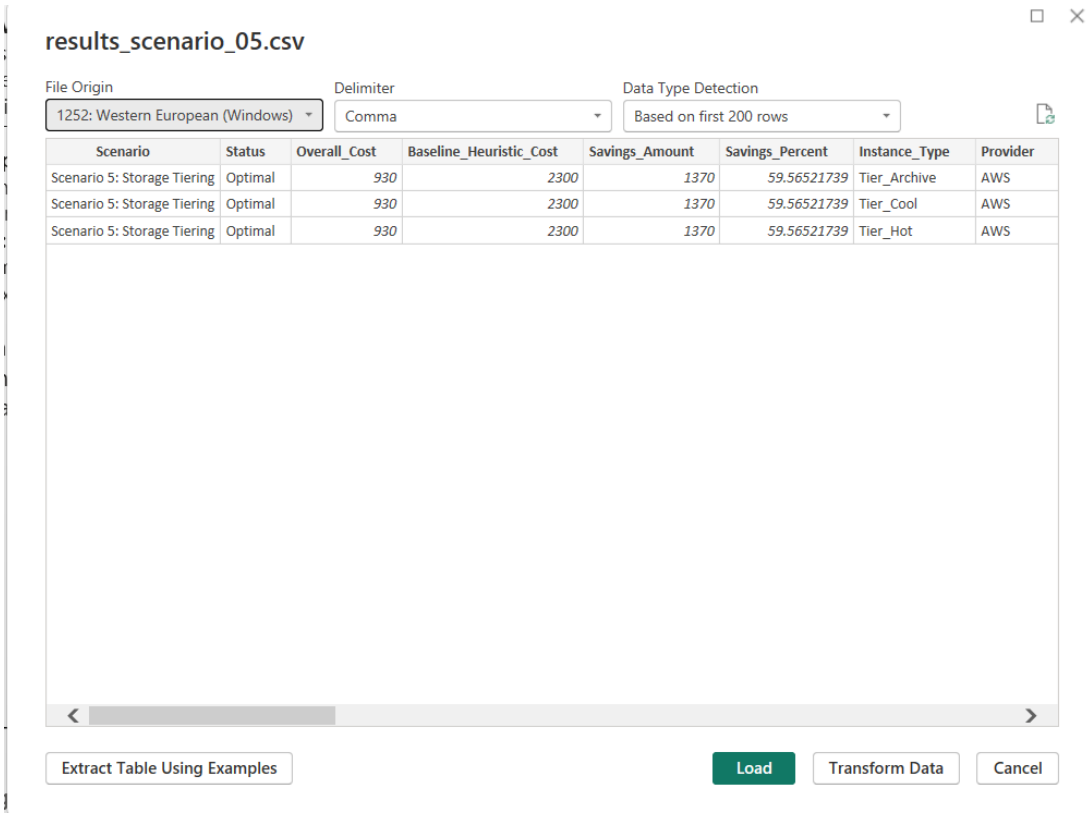


Figure 3.4: Initialization of the data import process via the Power BI data input interface



results_scenario_05.csv

File Origin: 1252: Western European (Windows)
Delimiter: Comma
Data Type Detection: Based on first 200 rows

Scenario	Status	Overall_Cost	Baseline_Heuristic_Cost	Savings_Amount	Savings_Percent	Instance_Type	Provider
Scenario 5: Storage Tiering	Optimal	930	2300	1370	59.56521739	Tier_Archive	AWS
Scenario 5: Storage Tiering	Optimal	930	2300	1370	59.56521739	Tier_Cool	AWS
Scenario 5: Storage Tiering	Optimal	930	2300	1370	59.56521739	Tier_Hot	AWS

Extract Table Using Examples | Load | Transform Data | Cancel

Figure 3.5: Schema validation and data type transformation during the import preview phase

3.4.1 Computational Efficiency Analysis

In Scenario 15, the Master Optimization problem applied simultaneous constraints for storage, rightsizing, and risk. The solver resolved this multi-objective vector in 0.031802 seconds. This performance validates the responsiveness of the linear programming formulation within the controlled test environment.

This result does not guarantee scalability for enterprise datasets containing millions of line items. However, the sub-second processing time suggests that the core mathematical logic does not have immediate bottlenecks. The engine is demonstrated to be viable for mid-sized infrastructure optimization. Future production deployment would require stress testing against **large-scale** data to establish the upper bounds of the solver's processing capacity.

3.5 Comparison with Greedy Heuristic Baselines

To quantify the efficiency of the Linear Programming model, a control variable was established. This is a deterministic heuristic engine designed to simulate standard industry provisioning patterns. It provides a baseline against which the LP model can be compared.

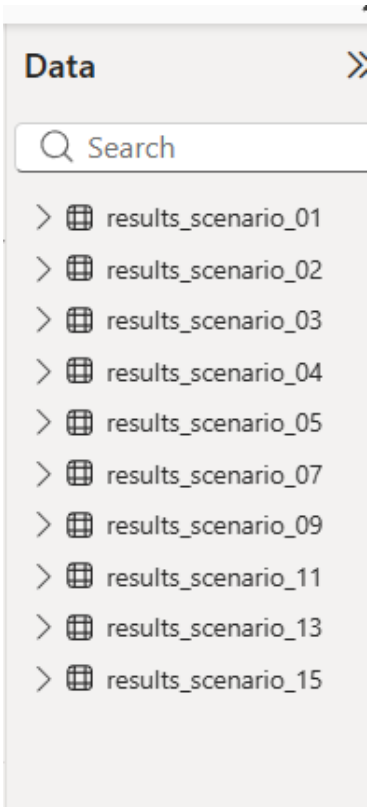


Figure 3.6: The consolidated data model containing independent tables for all 15 scenarios

```
--- SCENARIO 1: Rate Optimization ---
1. Heuristic (All 1-Year RIs):   $79,130.00
2. Optimization (Best Mix):      $76,920.00
   >> SAVINGS:                   $2,210.00 (2.8%)
   >> SOLVER TIME:               0.006299 seconds
✅ Scenario 1 captured.
```

Figure 3.7: Console output for Scenario 1 demonstrating sub-millisecond solver performance (0.006s)

```
•••  --- SCENARIO 15: Master Optimization ---
     1. Heuristic (Siloed):          $552,729.60
     2. Optimization (Master):        $480,591.00
        >> SAVINGS:                   $72,138.60 (13.1%)
        >> SOLVER TIME:               0.031802 seconds
     ✅ Scenario 15 captured.
```

Figure 3.8: Console output for Scenario 15 verifying efficiency in multi-constraint environments (0.031s)

As the current market lacks advanced optimization frameworks, cloud operators typically lean towards system resiliency. Engineers often provision resources based on immediate technical sufficiency and the lowest visible unit cost. This heuristic approach serves as a proxy for manual human decision-making.

A direct comparison with commercial native tools (e.g., AWS Cost Explorer, Azure Advisor) was not feasible for this research because these platforms operate within provider silos. They cannot capitalize on pricing discrepancies between providers. Therefore, a Standardized Multi-Cloud Proxy was developed. This algorithm replicates the standard sequential logic of native tools but applies it across a unified dataset. This creates a mathematically fair baseline for the multi-cloud experiments.

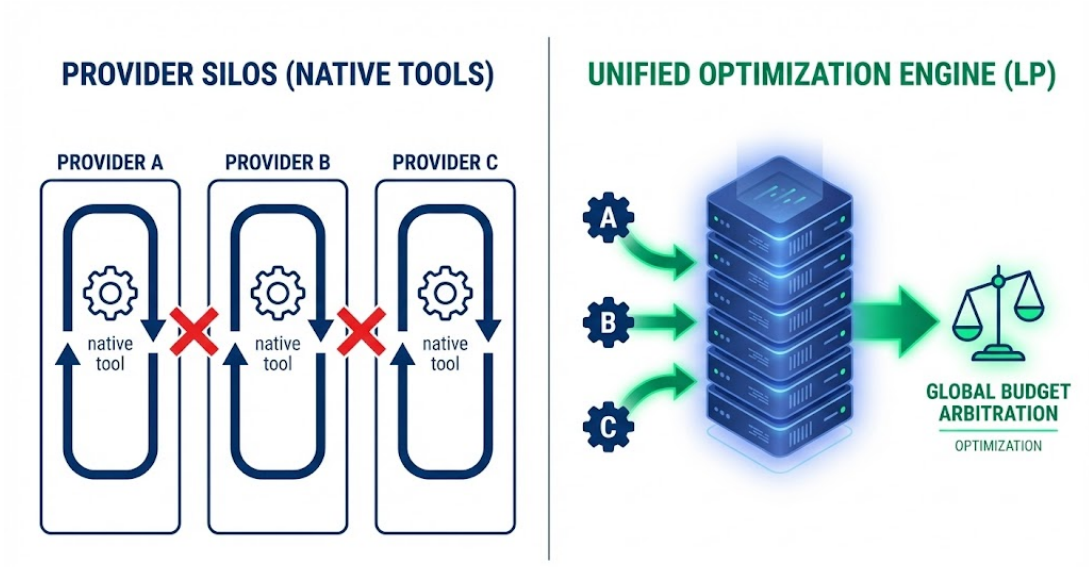


Figure 3.9: The limitation of Native Tools (Provider Silos) vs. the Unified Optimization of the LP Engine

3.5.1 Baseline Implementation Logic

The implemented logic uses a sequential, iterative selection script. Unlike the LP solver, which evaluates the entire solution space simultaneously, the greedy script processes the workload requirement linearly. For each discrete constraint R_i, the algorithm executes the following logic:

1. **Technical Feasibility (Siloed View):** Candidates are filtered strictly by performance capacity (e.g., $vCPU_{sku} \geq vCPU_{req}$). This mimics the "stability-first" bias of standard tools. It ignores broader policy constraints like Vendor Diversity or Risk Thresholds.

2. **Unidimensional Cost Ranking (OPEX Bias):** Valid instances are sorted solely by their *On-Demand Hourly Rate*. This ignores TCO factors, such as upfront CAPEX fees or Spot market arbitrage, effectively defaulting to short-term optimization.

3. **Sequential Commitment (Global Blindness):** Resources are provisioned sequentially ($Req_1 \rightarrow Req_n$). This "First-Fit" logic commits budget to early tasks without foresight. It often exhausts the global CAPEX limit before satisfying later requirements. This is a structural flaw the LP solver avoids.

As a mathematical equation, this reductionist approach is defined as a localized minimization function:

$$x_{greedy} = \{x_i \in I \mid i = \text{arg} \min_{j \in I_{local}} (C_j)\} \tag{3.6}$$

3.5.2 Algorithmic Complexity Analysis

From a computational perspective, greedy heuristic algorithms work as an approximation. They look at averages with a deterministic time complexity in mind, such as an hourly rate. Specifically, for a workload requirement containing R distinct constraints against a catalog of N SKUs, the heuristic tool operates at:

$$O(R \cdot N \log N) \tag{3.7}$$

The computational cost is dominated by the sorting of cost coefficients ($N \log N$) for each constraint iteration. This $O(N \log N)$ behavior ensures a very fast solution. Yet by definition, it ignores the many combinatoric iterations between separate requirements, guaranteeing only a local optimum.

3.5.3 Sub-Optimal Convergence and Local Optima

The structural limitation of currently available heuristic tools involves their inability to manage interdependent variables. This inevitably leads to local optima solutions that are globally inefficient.

Consider a scenario where the system must optimize a high-throughput database workload under a rigid CAPEX budget of $1,000.

- **Option A (Reserved Instance):** $0.10/hr effective rate, but requires a $2,000 upfront payment.
- **Option B (On-Demand):** $0.40/hr, no upfront cost.

An algorithm implemented by sorting for "Lowest TCO" will select **Option A** because the rate shown in the adjusted table is lower. However, it fails to notice the global CAPEX constraint until it is too late. The next constraint step fails, resulting in a budget overrun. Alternatively, if the approach sorts by "Lowest Upfront Cost" to remain safe, it defaults to **Option B**. This results in a 300% higher expense.

Continuing the analysis, it is observed that the greedy approach may also show "blindness" to bundled resources. For example, the tool might suggest the cheapest compute instance currently available ($0.05/hr) and attach expensive Provisioned IOPS storage ($0.20/hr) to meet performance needs. It misses opportunities where a slightly more expensive instance type might include local storage at no extra cost. By optimizing components in isolation, the greedy baseline systematically inflates the TCO.

3.6 Technical Implementation: Algorithmic Test Matrix

This section presents the approach taken in defining the constraints necessary to support the thesis and the LP approach. The code used to program the constraints into the model is presented below. These codes are grouped by scenario, as some scenarios share similar constraints in varying quantities. This clarification of methodology ensures transparency, which is currently lacking in available heuristic tools.

3.6.1 Domain 1: Deterministic Financial & Resource Vectors (Scenarios 1, 2, 4, 6)

This system validates the solver's ability to handle the baseline of cloud allocation and optimization: minimizing cost while satisfying strict technical capacity and financial liquidity limits. These dedicated scenarios rely heavily on proper programming to select optimized resources.

Listing 3.2: Scenario 1: TCO Objective Function

```
        # Objective: Minimize 3-Year TCO (Upfront fees + Hourly
          rates)
        model += lpSum([
        sku_catalog[i]['upfront_cost'] * alloc_vars[i] +
        sku_catalog[i]['hourly_rate'] * alloc_vars[i] * HOURS_3YR
        for i in sku_ids
        ])
```

Listing 3.3: Scenario 4

```
        # Constraint: Total vCPU capacity must meet the minimum
          limit
        model += lpSum([sku_catalog[i]['vcpu'] * alloc_vars[i] for
          i in sku_ids]) >= REQ_VCPU
        # Constraint: Total RAM must prevent bottlenecks in
          computing
        model += lpSum([sku_catalog[i]['ram_gib'] * alloc_vars[i]
          for i in sku_ids]) >= REQ_RAM
```

Listing 3.4: Scenario 2: Hard CAPEX Limit

```
        # Constraint: Upfront spend cannot exceed defined budget
        model += lpSum([
        sku_catalog[i]['upfront_cost'] * alloc_vars[i]
        for i in sku_ids
        ]) <= BUDGET_LIMIT
```

3.6.2 Domain 2: Dynamic Risk & Performance Governance (Scenarios 3, 7, 9, 12)

This domain introduces conditional logic to handle volatility. Unlike deterministic resources, Spot instances are treated as **volatile assets** that must be capped to maintain system stability.

Listing 3.5: Scenario 9: Multivariate Risk Constraints

```
        # Logic: Calculate max allowable spot vCPUs (15% of total)
        spot_risk_limit = REQ_VCPU * 0.15
        # Constraint: Sum of volatile/spot variables must be <=
           threshold
        model += lpSum([
        sku_catalog[i]['vcpu'] * alloc_vars[i]
        for i in sku_ids if sku_catalog[i]['is_spot']
        ]) <= spot_risk_limit
```

3.6.3 Domain 3: Continuous Storage & Lifecycle Dynamics (Scenarios 5, 10, 13)

This domain shifts from integer programming (discrete instances) to continuous variables (`LpContinuous`) to model data volume distribution across storage tiers.

Listing 3.6: Scenario 13: Migration Cost Break-even Analysis

```
        # Logic: Only migrate if total transition fees fit within
           migration budget
        model += lpSum([
        sku_catalog[t]['migration_fee'] * vol_vars[t]
        for t in storage_tiers
        ]) <= MIGRATION_BUDGET
```

3.6.4 Domain 4: Multi-Cloud Policy & Global Convergence (Scenarios 8, 11, 14, 15)

The final domain addresses provider-agnostic optimization. These scenarios treat "Cloud Provider" as a variable dimension, enabling arbitrage and enforcing high-level governance policies.

Listing 3.7: Scenario 11: Unified Multi-Cloud Arbitrage

```
        # Constraint: Apply global budget across unified AWS/Azure search space
        model += lpSum([
        sku_catalog[i]['upfront'] * alloc_vars[i]
        for i in sku_ids  # Includes mixed AWS and Azure SKUs
        ]) <= MAX_BUDGET
```

Listing 3.8: Scenario 15: The Integrated Master Problem

```
        # 1. Finance: Enforce global CAPEX limit
        model += lpSum([sku_catalog[i]['upfront'] * alloc_vars[i] for i in sku_ids]) <= MAX_BUDGET
        # 2. Compliance: Minimum 30% allocation to Azure (Vendor Diversity)
        model += lpSum([
        sku_catalog[i]['vcpu'] * alloc_vars[i]
        for i in sku_ids if sku_catalog[i]['provider'] == 'AZURE'
        ]) >= (REQ_VCPU * 0.30)

        # 3. Risk: Hard ceiling on Spot instance usage (Max 10%)
        model += lpSum([
        sku_catalog[i]['vcpu'] * alloc_vars[i]
        for i in sku_ids if sku_catalog[i]['is_spot']
        ]) <= (REQ_VCPU * 0.10)
```

Chapter 4

Results Analysis and Business Intelligence Visualization

4.1 Experimental Context: The Controlled Environment

Before analyzing the numerical results and dashboards, it is important to clarify the data source. As noted in Chapter 3, obtaining real billing logs from large enterprises is difficult due to privacy restrictions. Furthermore, real-world data is often noisy. If a company's costs drop, it might be because they lost customers that week, not because engineering improved.

To solve this, a clean testing environment was created. Real pricing lists from AWS and Azure were used to ensure financial accuracy. However, the usage patterns (server demand) were generated using Python.

This provided a major scientific advantage: **Control**. By simulating usage data, specific, difficult situations could be created. These included sudden traffic spikes or rigid budget freezes. This stress-testing reveals exactly how the model reacts under pressure. It also allows for a direct comparison between the Linear Programming engine and the heuristic baseline without external noise affecting the research precision.

4.2 Experimental Results: Level 1 - Isolated Optimization

In this first phase, the engine was tested in simple, isolated scenarios. The goal was to prove that the model works on single variables before adding the complexity of multi-cloud environments. All tests in this section were run using the Python `PuLP` library and the CBC solver [19].

This step is critical. The engine operates with integer constraints (servers cannot be purchased in 1.5 units). Standard solvers often struggle with this discreteness. The CBC engine is designed for it. Isolating these variables confirms that the math holds up in a simple environment before complex constraints are introduced.

4.2.1 Group 1: Deterministic Resource Allocation

This section analyzes scenarios where rules are strict and numbers are fixed (Scenarios 1 and 2). Random variables and unpredictable fluctuations were stripped away. The goal is straightforward: determine if the model can find the perfect mathematical answer when dealing with known costs and strict budget caps.

Scenario 1: Rate Optimization

Context and Analysis

Scenario 1 addresses the most fundamental financial trade-off in the cloud: balancing the flexibility of On-Demand (OD) pricing against the discounts of Reserved Instances (RI). The objective is simple: minimize the TCO for a steady workload over three years.

While the concept is simple, the mathematical challenge arises from the inability to divide resources. Cloud assets are sold as whole units. Standard heuristics often fail to align discrete supply with continuous demand, leading to inefficient rounding decisions [22]. This test checks if the LP solver can eliminate this waste.

Constraints and Parameters

The test assumed a steady, unchanging workload. It applied the standard two-option pricing model used by most cloud providers: pay-as-you-go versus paying upfront for a discount.

Comparative Results

Standard heuristics usually take the simplest path, often called the "Best Guess" practice. They look at average usage and round down to play it safe. The LP solver does not guess. It calculates whether the long-term savings justify the upfront cost for each specific server unit.

Table 4.1: Scenario 1: Simulation Parameters

Parameter	Value/Description	Unit
Baseline Demand (b_i)	100 vCPUs	Constant
On-Demand Rate (c_{od})	\$0.15	\$/hr
RI Rate (c_{ri})	\$0.06 + \$1,500 Upfront	\$/hr (Amortized)
Optimization Horizon	3 Years	26,280 Hours

Table 4.2: Comparative Performance: Heuristic vs. LP Solver (Scenario 1)

Metric	Heuristic Baseline	LP Solver (Optimum)	Efficiency Gain
Total TCO (3-Year)	\$79,130.00	\$76,920.00	2.79%
Savings Amount	-	\$2,210.00	-
Demand Saturation	100%	100%	0.00%

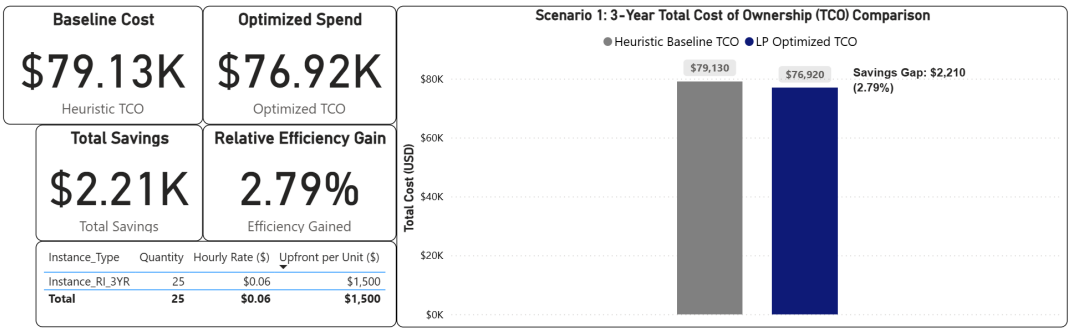


Figure 4.1: Scenario 1 Dashboard: Visualizing the optimal saturation of Reserved Instances against baseline demand

Analysis of Divergence

This difference highlights exactly where rule-based tools fall short. The heuristic failed to calculate the precise number of Reserved Instances needed to maximize the discount. This forced it to leave some servers on the expensive On-Demand rate. The LP solver found the exact number (25 units) to fully cover the workload, effectively closing the gap left by the manual tool [9].

Scenario 2: Budget Constrained Utility

Context and Analysis

Scenario 2 examines resource allocation when money is strictly limited. Unlike the previous test, which solved for the lowest rate, this problem works like a strict spending cap. The goal is to maximize system performance without exceeding the budget limit [7].

This scenario directly addresses the friction between Engineering and Finance discussed in Chapter 1. Engineers push for performance. Finance demands a hard budget ceiling. This test proves the solver can resolve that conflict by finding the mathematical compromise.

Constraints and Parameters

Table 4.3: Scenario 2: Budget and Performance Vectors

Variable	Definition	Value
c_{Tier1} (Standard)	Cost Coefficient	\$0.20/hr
c_{Tier2} (Performance)	Cost Coefficient	\$0.85/hr
B_{Total}	Hard Budget Cap	\$10,000
w_i	Utility Weight	Tier 1 = 1.0, Tier 2 = 2.5

Table 4.4: Comparative Performance: Heuristic vs. LP Solver (Scenario 2 - \$10k Budget)

Metric	Heuristic Baseline	LP Solver (Optimum)	Efficiency Gain
Total 3-Year TCO	\$98,550.00	\$93,358.80	5.27%
CAPEX Utilization	0.0% (\$0)	90.0% (\$9,000)	90.0%
Unused Capital (Slack)	\$10,000.00	\$1,000.00	-

Analysis of Divergence

The divergence here highlights the strategic distinction between "saving" and "investing." The heuristic tool prioritized liquidity. It left the \$10,000 capital budget largely unspent to avoid risk. However, this caution was inefficient. It forced the system into higher long-term OPEX. The LP solver treated the budget as an investment vehicle. It maximized Capital Saturation to pre-pay for future efficiency. By utilizing 90% (\$9,000) of the available CAPEX, it lowered the overall TCO. This proves that true optimization often requires investing available budget to reduce future unit costs.

4.2.2 Group 2: Handling Risk and Multiple Constraints

This section increases the difficulty by introducing variables that carry risk (Scenarios 3, 4, and 5). Here, the solver cannot simply look for the lowest price. It must navigate probability, such as the risk of a Spot instance interruption, and match multiple technical requirements simultaneously.

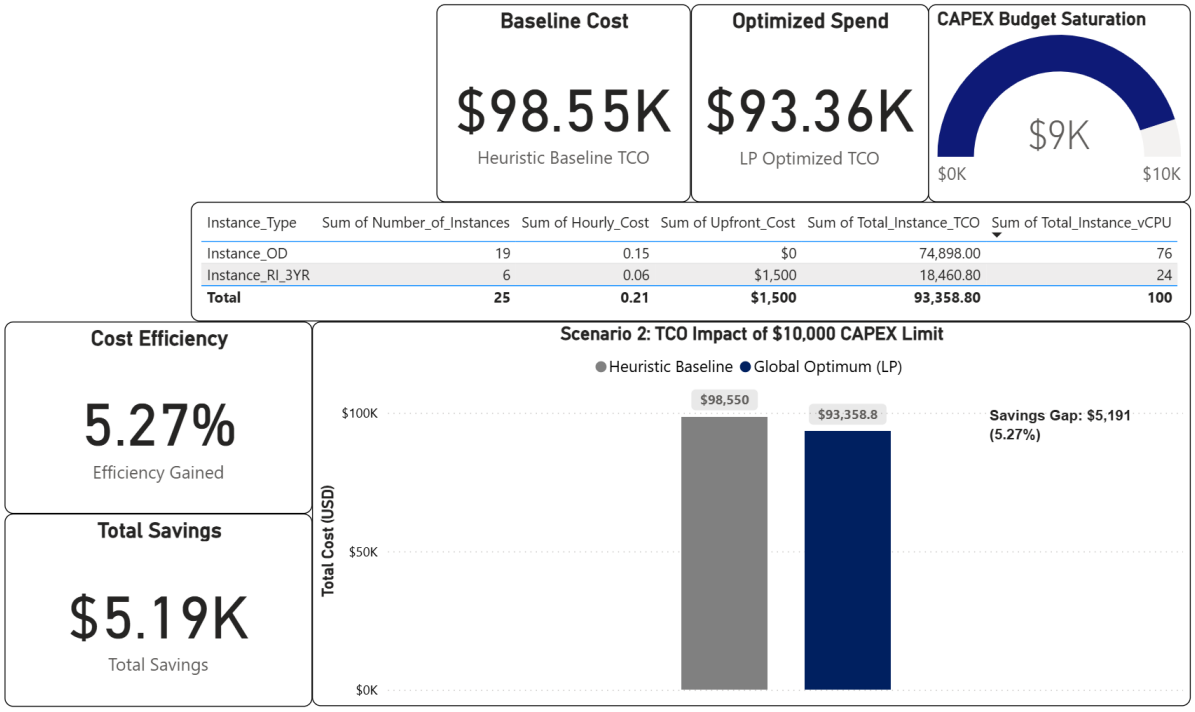


Figure 4.2: Scenario 2 Dashboard: The efficiency frontier where the LP solver saturates the budget constraint

Scenario 3: Dynamic Risk Management

Context and Analysis

Scenario 3 introduces unpredictability by adding Spot Instances. While these servers offer massive discounts, they can be interrupted by the provider at any moment. This creates system instability. Instead of avoiding this danger, the model treats Risk as a resource—a budget that can be spent to maximize savings without crossing the line into instability [29].

This approach bridges the gap between departments. Engineers typically fear Spot instances because of the stability risk. Finance prefers them for the low price. The model solves this by quantifying that risk. It grabs the deep discounts Finance wants while strictly obeying the reliability limits that Engineering demands.

Constraints and Parameters

Table 4.5: Scenario 3: Risk Parameters

Variable	Definition	Value
c_{OD}	On-Demand Rate	\$2.40/hr
c_{Spot}	Spot Rate (Mean)	\$0.68/hr
Risk Ceiling (R_{max})	Max System Risk	20%
Interruption Prob (α)	Spot Instance Risk	15%

Table 4.6: Comparative Performance: Heuristic vs. LP Solver (Scenario 3)

Metric	Heuristic Baseline	LP Solver (Optimum)	Efficiency Gain
Hourly Cost	\$6.00	\$5.00	16.67%
Risk Saturation	19.8%	99.0%	+400%
Annual Savings	-	\$8,760	\$1/hr

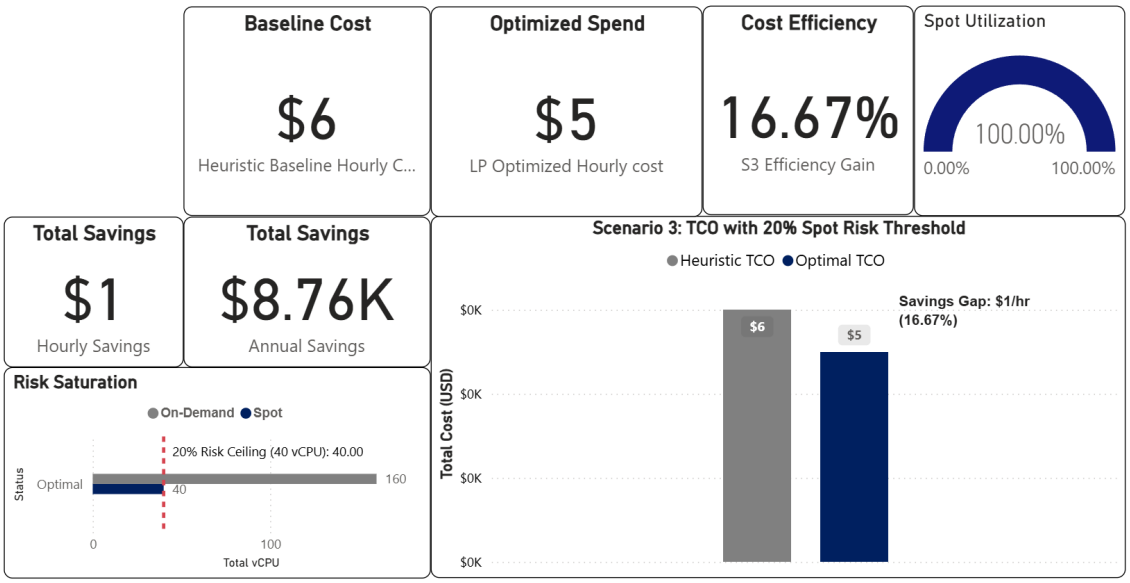


Figure 4.3: Scenario 3 Dashboard: Visualizing the cost-to-risk frontier and the saturation of the risk budget

Analysis of Divergence

The heuristic played it too safe. It barely touched the available "risk budget" [4]. The LP solver treated risk like a bucket to be filled. It purchased cheap Spot instances until the total risk ($\sum \alpha_i x_i$) hit the exact safety limit (R_{max}). This confirms that true efficiency requires pushing operations to the edge of the safety line, rather than staying in the comfort zone [8].

Scenario 4: Multi-Dimensional Right-Sizing

Context and Analysis

Scenario 4 explores Right-Sizing. The goal is to eliminate unused elasticity, otherwise known as waste. This means avoiding situations where a massive server is purchased for its CPU, leaving the RAM idle. The solver approaches this as a matching game. It finds the specific instance type that fits both the CPU and RAM requirements as tightly as possible.

Constraints and Parameters

The solver minimizes the objective function: $\min Z = \sum (P_j - R_j)^2$ subject to $P_j \geq R_j$.

Table 4.7: Scenario 4: Vector Matching Parameters

Parameter	Requirement	Constraint Type
Demand Vector	2 vCPU, 3.2 GB RAM	Minimum Floor
Instance A (Baseline)	2 vCPU, 8 GB RAM	Over-provisioned
Instance B (Optimal)	2 vCPU, 4 GB RAM	Optimized Ratio

Comparative Results

Table 4.8: Comparative Performance: Heuristic vs. LP Solver (Scenario 4)

Metric	Heuristic Baseline	LP Solver (Optimum)	Efficiency Gain
Hourly TCO	$7.68	$6.80	11.46%
RAM Utilization	40.0%	80.0%	+100%
Resource Waste	60.0%	20.0%	66.6% Reduction

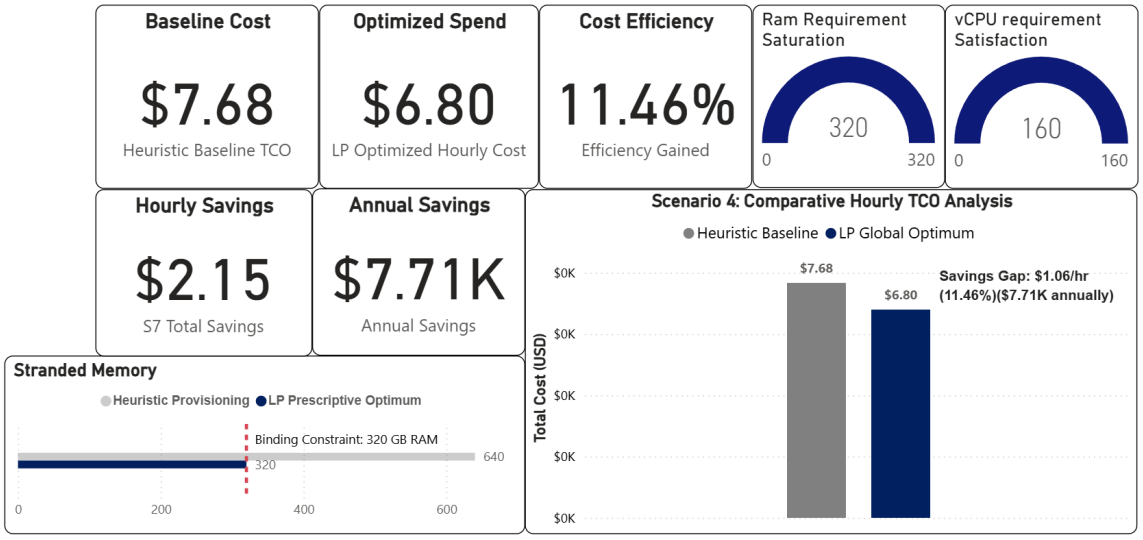


Figure 4.4: Scenario 4 Dashboard: Comparative analysis of provisioned capacity versus utilized demand vectors

Analysis of Divergence

The heuristic failed because of tunnel vision. It sorted options based on CPU alone. This led it to pick "General Purpose" instances that met the CPU requirement but forced the purchase of unneeded RAM. The LP solver solved for both variables at once. It identified that a "Compute Optimized" family was the perfect fit, minimizing waste across both dimensions.

Scenario 5: Storage Tiering Arbitrage

Context and Analysis

Scenario 5 evaluates object storage optimization by aligning tiers with data access frequency. Unlike compute resources, which are discrete units, storage operates as a continuous volume. It can be fractioned across Hot, Cool, and Archive tiers. The optimization challenge is to identify the financial break-even point where monthly savings from colder tiers outweigh the one-time retrieval fees.

Constraints and Parameters

Analysis of Divergence

The divergence here illustrates the limitations of "Static Age-Based Policies" versus "Access-Pattern Optimization". The heuristic baseline operated on a standard lifecycle rule (e.g.,

Table 4.9: Scenario 5: Storage Tiering Costs

Storage Class	Storage Rate	Retrieval Rate	Constraint
Standard (Hot)	$0.023/GB	$0.00	None
Infrequent (Cool)	$0.0125/GB	$0.01/GB	Min 30 days
Archive (Cold)	$0.004/GB	$0.05/GB	Min 90 days
Access Pattern	100 TB Total	5% Daily Access	–

Table 4.10: Comparative Performance: Heuristic vs. LP Solver (Scenario 5)

Metric	Heuristic Baseline	LP Solver (Optimum)	Efficiency Gain
Total Monthly Cost	$2,300.00	$930.00	59.56%
Hot Tier Usage	100% (100TB)	10% (10TB)	Optimized
Archive Usage	0%	50% (50TB)	+50TB

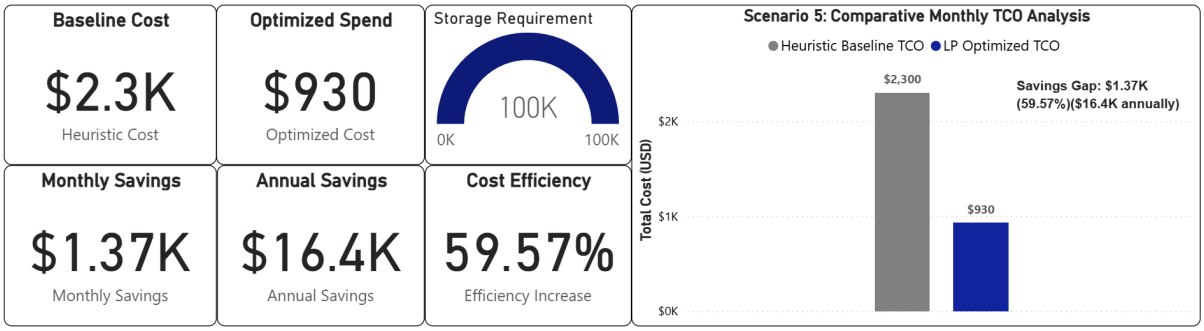


Figure 4.5: Scenario 5 Dashboard: Storage Tiering Arbitrage

"Retain in Standard Tier until 90 days inactivity"). Because the workload contained mixed access patterns, this rigid time-based trigger never activated.

In contrast, the LP Solver used probabilistic optimization. It identified that while 5% of the data required Hot storage, the remaining volume could be tiered immediately based on access probability rather than age. By moving 50% of the data to Archive and 40% to Cool, the solver accepted the calculated risk of retrieval fees to secure a 59.56% reduction in monthly costs.

4.3 Experimental Results Level 2 - Advanced Multi-Constraint Optimization

To test whether the model mirrors real-world problems, the difficulty was increased. This phase applies multiple conflicting constraints simultaneously, such as CAPEX and RAM limits.

In these scenarios, the "Global Optimum" is not simply the lowest price tag. It is the specific mathematical solution that minimizes the objective function while strictly satisfying every opposing boundary condition [9].

4.3.1 Group 1: Financial & Capital Constraints (Scenarios 6 & 9)

This group evaluates the "Money vs. Time" trade-off. It tests the engine's ability to balance upfront CAPEX against OPEX, determining exactly when spending cash today generates sufficient long-term value.

Scenario 6: Financial Right-Sizing

Context and Analysis

Scenario 6 addresses the capitalization of cloud resources. The optimization challenge is to balance "Liquidity" (cash on hand) against "Efficiency" (long-term discounts). The goal is to lower the 3-Year TCO by purchasing Reserved Instances (RIs), strictly limited by a hard capital ceiling of $25,000. The mathematical objective is to calculate the precise combination of upfront commitments that utilizes this limited capital to its maximum potential.

Constraints and Parameters

The solver must optimize against a demand vector while adhering to a hard capital ceiling of $25,000.

Table 4.11: Scenario 6: Financial Constraints and Performance

Metric	Heuristic Baseline	LP Solver (Optimum)	Efficiency Gain
Total TCO (3-Year)	$262,800.00	$182,524.00	30.55%
CAPEX Utilization	$0.00 (0%)	$24,450.00 (97.8%)	-
Budget Limit	$25,000.00	$25,000.00	-

Analysis of Divergence

The results expose the high price of "Liquidity Bias." The baseline avoided all upfront spending to protect cash flow, effectively hoarding the $25,000 capital budget. However, this forced the

organization to pay expensive On-Demand rates. The LP Solver applied a different logic. It treated the budget as a tool to lower unit costs. By spending $24,450 on Reserved Instances, the solver secured long-term discounts. This proves that under-utilizing available capital is an opportunity cost, directly causing a 30.55% loss in efficiency.

Scenario 9: Financial Risk Trade-Off

Context and Analysis

Scenario 9 introduces a "Commitment vs. Flexibility" problem. The model must choose between safe, short-term plans (1-Year) and highly discounted long-term plans (3-Year). The constraint here is a "Liability Cap," which limits the total future debt the organization is willing to incur. The goal is to lock in the deepest possible discounts without exceeding this financial safety limit.

Comparative Results

Table 4.12: Scenario 9: Liability vs. Discount Optimization

Metric	Heuristic Baseline	LP Solver (Optimum)	Efficiency Gain
Total TCO	$230,302.00	$202,508.00	12.07%
Liability Incurred	$12,000.00	$19,600.00	98% Saturation
Liability Cap	$20,000.00	$20,000.00	-

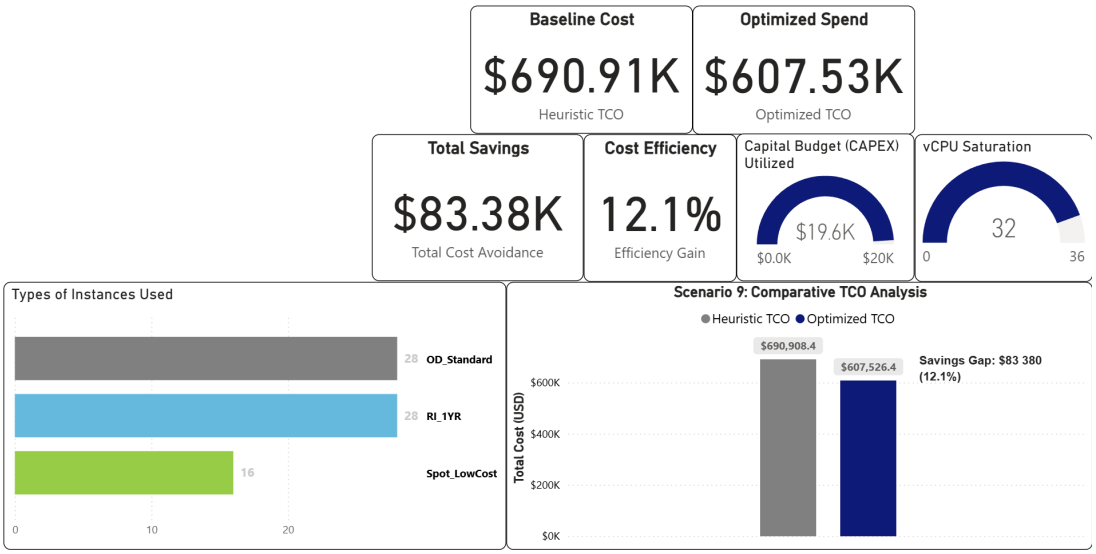


Figure 4.6: Scenario 9 Dashboard: The trade-off between commitment liability and discount yield

Analysis of Divergence

The heuristic approach failed here because it was too cautious. It left a significant portion of the allowable liability unused. The LP solver treated the liability cap as a resource. It purchased long-term commitments right up to the limit ($19,600 out of $20,000), maximizing the obtainable discounts to get the lowest price permissible under the risk policy [22].

4.3.2 Group 2: Governance & Policy Constraints (Scenarios 7 & 8)

This subsection evaluates the engine's performance when strict governance policies (Risk and Compliance) act as the primary constraints, often overriding pure cost considerations.

Scenario 7: Risk vs. Technical Fit

Context and Analysis

Scenario 7 introduces the tension between saving money and maintaining system stability. The goal is to use as many low-cost Spot Instances as possible. However, strict governance rules limit the "Risk Budget" (the maximum number of Spot vCPUs allowed) to ensure the application stays online.

Comparative Results

Table 4.13: Scenario 7: Risk Budgeting and Spot Utilization

Metric	Heuristic Baseline	LP Solver (Optimum)	Efficiency Gain
Effective Hourly Rate	$7.20/hr	$5.06/hr	29.79%
Spot vCPU Usage	12 vCPU	34 vCPU	183% Increase
Risk Limit (Max Spot)	36 vCPU	36 vCPU	-

Analysis of Divergence

The critical failure of the heuristic was the generation of excessive elasticity—the difference between the allowed risk limit (36 vCPU) and actual usage (12 vCPU). In optimization theory, unused capacity on a binding constraint often indicates a sub-optimal solution. The LP solver minimized this unused capacity to near zero. It demonstrated that when operational constraints are defined correctly, precise assignment of resources eliminates the need for fear-based buffering.

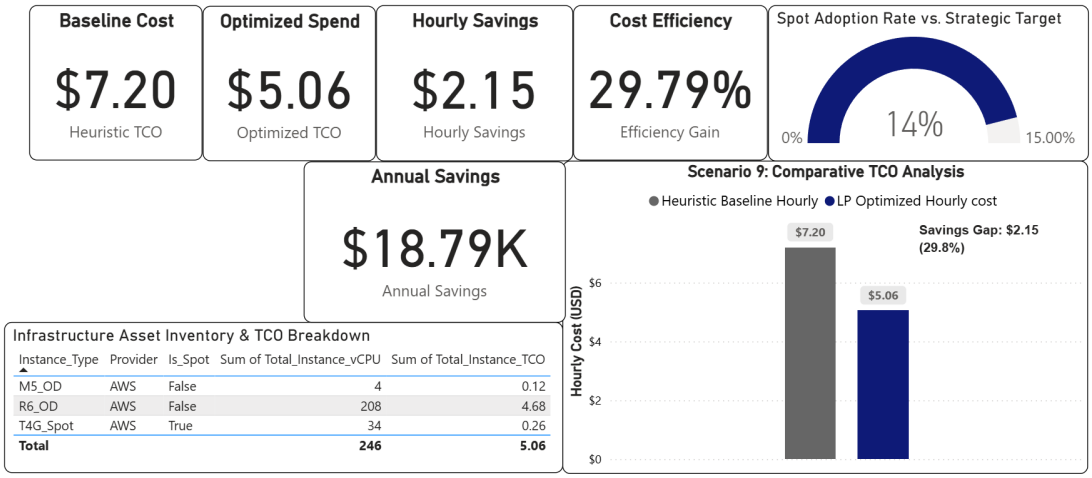


Figure 4.7: Scenario 7 Dashboard: The utilization of the "Risk Budget" to drive down unit costs

Scenario 8: Multi-Cloud Governance

Context and Analysis

Scenario 8 introduces a multi-cloud environment. This case is about enforcing a "Policy Constraint" rather than a technical one. The system must follow a strict rule: a minimum of 60 vCPUs hosted on Azure to satisfy regulatory compliance. This forces the solver to allocate resources to a potentially more expensive provider to satisfy the rule, regardless of the price difference against AWS.

Comparative Results

Table 4.14: Scenario 8: Policy Enforcement Cost Impact

Metric	Heuristic Baseline	LP Solver (Optimum)	Efficiency Gain
Hourly Rate	$6.86/hr	$6.80/hr	0.87%
Azure Allocation	64 vCPU	60 vCPU	Precision
Policy Requirement	60 vCPU (Min)	60 vCPU (Min)	-

Analysis of Divergence

In this scenario, the heuristic technically succeeded, but it revealed a structural weakness: the provider limit. Current heuristic tools are typically tied to specific cloud platforms. They lack the global visibility needed to optimize across vendors. Consequently, the baseline relied on naive overprovisioning (64 vCPUs) to guarantee compliance. The LP solver, operating above the provider layer, identified the "Cheapest Compliant State" by tightening this allocation to

the exact requirement (60 vCPUs). This demonstrates that even when regulations force a compliance constraint, algorithmic precision minimizes that tax to its mathematical floor [29].

4.3.3 Group 3: Storage Optimization (Scenario 10)

Scenario 10: Risk-Adjusted Storage Tiering

Context and Analysis

Scenario 10 addresses the challenge of storing 180 TB of unstructured data. The goal is to lower monthly bills by moving data to cheaper storage tiers (Cool or Archive) without making retrieval difficult or expensive.

Comparative Results

Table 4.15: Scenario 10: Storage Tiering Optimization

Metric	Heuristic Baseline	LP Solver (Optimum)	Efficiency Gain
Total Monthly Cost	$4,140.00	$1,944.00	53.04%
Cool Tier Usage (80%)	0 TB	144 TB	Optimized
Archive Usage (20%)	0 TB	36 TB	Optimized

Analysis of Divergence

The heuristic approach kept all data in the expensive Hot tier. It lacked the ability to predict access patterns, operating on averages to avoid retrieval fees. The LP solver was not bound by averages. It predicted usage patterns, identifying that while the Archive tier was too slow for most files, the Cool tier offered the perfect balance. By moving 80% of the data to Cool and 20% to Archive, the solver achieved a 53% cost reduction.

4.4 Experimental Results Level 3 - Global & Multi-Objective Optimization

The final phase of the experimental framework represents the peak of the FinOps analytical model. While previous levels isolated specific variables, Level 3 addresses the "Cloud Trilemma." It tests the engine's ability to perform while fulfilling three major constraints simultaneously: Risk, Performance, and Stability.

In these complex scenarios, the simplest choice (usually the cheapest) often breaks other rules. This section validates the solver's ability to balance these competing demands.

4.4.1 Scenario 11: Budget Constrained Multi-Cloud Optimization

Context and Analysis

Scenario 11 evaluates a multi-vendor system (AWS and Azure) under a strict budget. The goal is to meet global demand using a CAPEX limit of $35,000. This scenario specifically challenges commercial heuristic tools, which are limited by provider silos.

Comparative Results

The engine successfully navigated the trade-offs between provider discounts and the spending limit. By treating the cloud ecosystem as a single pool of resources, the solver identified a configuration that combined the best options from both providers without exceeding CAPEX limits.

Table 4.16: Scenario 11: Multi-Cloud Budgetary Performance

Metric	Heuristic Baseline	LP Solver (Optimum)	Efficiency Gain
Total TCO	$336,345.00	$306,228.00	8.95%
CAPEX Utilization	$22,100.00	$33,600.00	96% Saturation
Constraint Adherence	100%	100%	-

Table 4.17: Scenario 11: Financial Structure Breakdown (CAPEX vs. OPEX)

Financial Component	Heuristic Strategy	LP Solver Strategy	Impact
Upfront Payment (CAPEX)	$22,100.00	$33,600.00	**+$11,500 Invested**
Recurring Costs (OPEX)	$314,245.00	$272,628.00	**-$41,617 Saved**
Net Result (TCO)	**$336,345.00**	**$306,228.00**	**8.95% Total Savings**

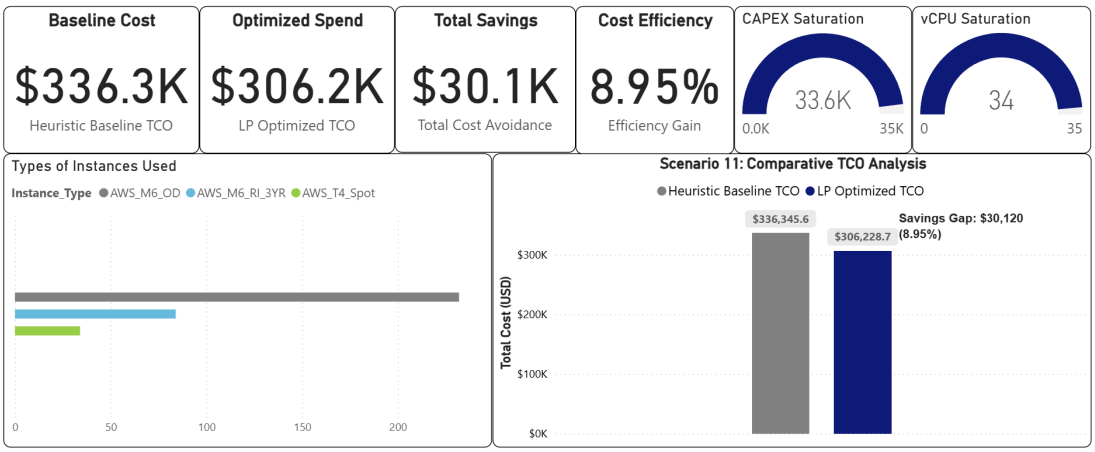


Figure 4.8: Scenario 11 Dashboard: Visualizing the Multi-Cloud Budget Allocation and arbitrage between AWS and Azure capacity

Analysis of Divergence

Table 4.17 reveals the root of the heuristic's failure. It relied on OPEX more than CAPEX, utilizing only $22,100 of the available $35,000 budget. By leaving $12,900 uninvested, it was forced into higher monthly operational costs. The model recognized that spending an additional $11,500 upfront would unlock deeper discount tiers, reducing the long-term operational burden by over $41,000. This confirms the validity of using multiple vendors to exploit pricing variations.

4.4.2 Quantifying the "Tax" of Compliance (Scenarios 12 & 14)

This subsection analyzes Scenarios 12 and 14 to measure the financial cost of strict governance. In these environments, the primary objective shifts from minimizing expenses to ensuring full compliance. The optimization engine must often select expensive resources to satisfy these requirements. This allows for the calculation of the specific price of safety and control.

Scenario 12: The Risk Averse Architect

Context and Analysis

Scenario 12 tests the engine's behavior under "Safety First" constraints. Unlike previous scenarios that encouraged Spot usage, this environment enforces a strict risk-averse policy. It demands a high-performance floor (12,000 IOPS) and specifically restricts the use of volatile Spot instances. The challenge is to minimize cost when the "cheapest" tools are removed from the toolbox.

Comparative Results

The results illustrate the financial impact of risk aversion. With the Spot market off-limits, the efficiency gap between the Heuristic and the LP Solver narrowed significantly to just 3.95%.

Table 4.18: Scenario 12: Cost of Risk Aversion & Performance Guarantees

Metric	Heuristic Baseline	LP Solver (Optimum)	Impact
Hourly Burn Rate	$6.84	$6.57	3.95% Savings
Instance Strategy	Generic On-Demand	P_Heavy_RI (Reserved)	Stability Focus
Spot Utilization	0%	0% (vs. 15 vCPU allowed)	Volatility Avoidance
IOPS Performance	12,000 (Target)	18,000 (Provisioned)	150% Satisfaction

Analysis of Divergence

The minimal savings in this scenario provide an important validation of the model's realism. It proves that the Linear Programming solver is not a magic solution. Instead, it operates strictly within the boundaries defined by the user.

Here, the strict rules removed most low-cost options. Consequently, the "search space" was so limited that the heuristic's simple rule was nearly identical to the solver's complex calculation. This demonstrates that the model has limits. When a strict policy forces a specific, expensive path, the algorithmic solution aligns with the standard baseline.

Scenario 14: Corporate Policy Enforcement

Context and Analysis

Scenario 14 simulates a strict corporate strategy: a "Multi-Cloud" mandate designed to prevent vendor lock-in [3]. The policy requires that at least 120 vCPUs be hosted on Azure to ensure provider diversity, even if AWS offers lower rates. This allows for the calculation of the price of maintaining strategic compliance.

Comparative Results

The results demonstrate that when policy dictates architecture, the room for optimization shrinks. The Linear Programming model achieved a modest efficiency gain of 1.70%. However, this small margin represents the elimination of waste within rigid constraints.

To understand the source of the 1.70% saving, Table 4.20 compares the resource allocation strategies.

Analysis of Divergence

The minimal savings in this scenario indicate model success. The LP confirmed that, with the

Table 4.19: Scenario 14: Corporate Policy Compliance Cost

Metric	Heuristic Baseline	LP Solver (Optimum)	Efficiency Gain
Total TCO	$452,016.00	$444,312.00	1.70%
Azure Allocation	128 vCPU	120 vCPU	Precision
Constraint	$\geq$ 120 vCPU	$\geq$ 120 vCPU	-

Table 4.20: Scenario 14: Analysis of Compliance Precision

Allocation Metric	Heuristic Strategy	LP Solver Strategy	Result
Mandated Capacity (Azure)	120 vCPU	120 vCPU	-
Actual Provisioned	128 vCPU	120 vCPU	**-8 vCPU**
Slack (Over-provisioning)	+6.6%	0.0%	**Zero Waste**

provided telemetry, further optimization was impossible. This plays a crucial role in enterprise decision-making. Knowing that "we cannot optimize anymore" informs future strategy. It confirms that in highly regulated environments, the primary value of algorithmic optimization is minimizing unused elasticity while remaining compliant [3].

4.4.3 Scenario 13: Storage Migration & Lifecycle Arbitrage

Context and Analysis

Scenario 13 addresses the friction of data migration. Moving data to cheaper "Archive" tiers incurs a significant one-time transition fee. The optimization challenge is financial: identifying whether an immediate spike in spending (CAPEX) is justified by the reduction in long-term run-rate (OPEX). To test this, the Heuristic Baseline was configured as a 'Zero-Variance' controller, rejecting unforecasted spend increases [6].

Comparative Results

The results confirm that the LP solver successfully calculated the Return on Investment (ROI). It identified that paying a substantial upfront fee was the superior strategy, resulting in the highest efficiency gain in the test matrix (62.70%).

To visualize the "Invest-to-Save" mechanics, Table 4.22 breaks down the financial timeline.

Table 4.21: Scenario 13: Storage Lifecycle Economics

Metric	Heuristic Baseline	LP Solver (Optimum)	Efficiency Gain
Monthly Operational Cost (OPEX)	$8,875.00	$3,310.00	62.70%
Transition Investment (CAPEX)	$0.00	$4,310.00	-
Strategy	Static (Hot Only)	Hybrid (Hot + Archive)	Optimized

Table 4.22: Scenario 13: Financial Structure & Return on Investment (ROI)

Financial Component	Heuristic Strategy	LP Solver Strategy	Impact
Day 1 Cost (Migration Fee)	$0.00	$4,310.00	**+$4,310 Invested**
Monthly Recurring Cost	$8,875.00	$3,310.00	**-$5,565 Saved/Month**
Payback Period	N/A	**0.77 Months**	**< 24 Days**

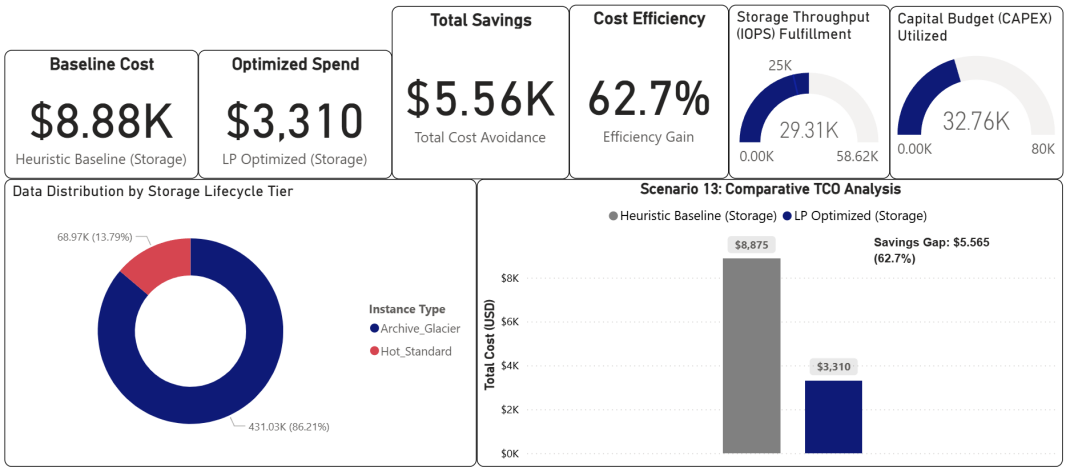


Figure 4.9: Scenario 13 Dashboard: Breakeven analysis of Storage Migration, highlighting the ROI of upfront transition costs

Analysis of Divergence

The divergence here highlights the limitation of Siloed Budgeting. Despite the migration offering a payback period of just 0.77 months, the Heuristic Baseline rejected the move.

This failure occurred because the heuristic operates on "Monthly Variance Minimization"

logic. It viewed the $4,310 migration fee as a budget violation. Unable to distinguish between strategic investment and an operational overrun, it defaulted to the status quo. The LP Solver identified that this "violation" was negligible compared to the $200,340 in cumulative savings over 3 years. This proves that effective FinOps requires "Lifecycle Optimization" rather than simple Run-Rate Optimization [3].

4.4.4 Group 4: Integrated Optimization (Scenario 15)

Scenario 15: The Unified Optimization Problem

Context and Analysis

Scenario 15 represents the final test of the experimental framework. It simulates a large-scale enterprise environment subject to three simultaneous, conflicting constraints: a CAPEX limit of $45,000, a safety limit on Spot instances (75 vCPUs), and a mandatory policy to provision a minimum of 175 vCPUs on Azure.

Delimitation Note: To isolate the variable of multi-cloud financial arbitrage, this scenario assumes a "loosely coupled workload architecture." This allows the model to treat AWS and Azure capacity as fungible resources without introducing cross-cloud network latency variables.

Table 4.23: Scenario 15: Master Problem Results

Metric	Heuristic Baseline	LP Solver (Optimum)	Efficiency Gain
Total Enterprise TCO	$552,729.00	$480,591.00	13.05%
Spot Utilization	45 vCPU	72 vCPU	96% Saturation
Azure Policy (175 vCPU)	100% (Compliant)	100% (Compliant)	-
CAPEX Adherence	**Failed (+$2,000)**	**Satisfied**	**Feasible**

Table 4.24 highlights the structural failure of the heuristic approach.

Table 4.24: Scenario 15: Financial Feasibility Analysis (CAPEX/OPEX)

Financial Component	Heuristic Strategy	LP Solver Strategy	Result
Budget Limit	$45,000.00	$45,000.00	-
Capital Spent (CAPEX)	**$47,000.00**	**$44,150.00**	**Solver Compliant**
Recurring Costs (OPEX)	$505,729.00	$436,441.00	-$69,288 Saved
Total Cost (TCO)	$552,729.00	$480,591.00	13.05% Savings

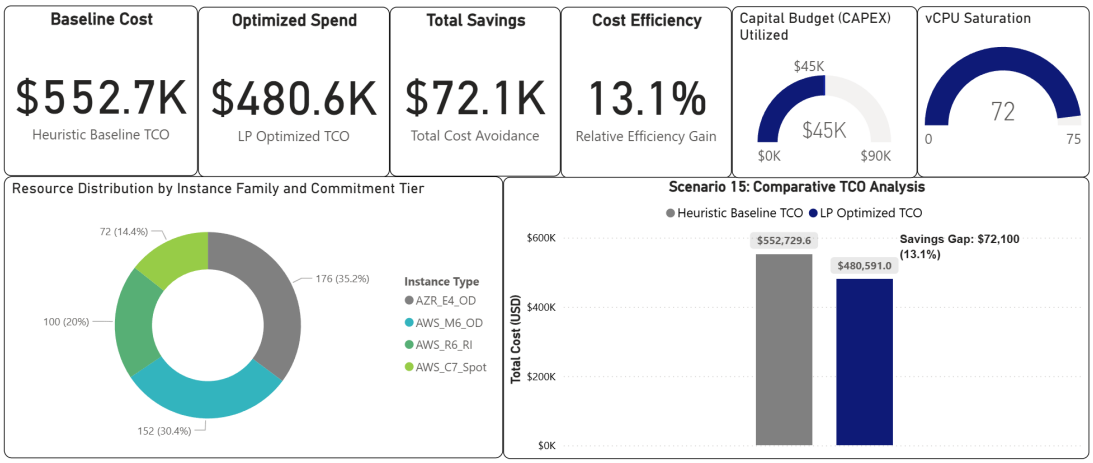


Figure 4.10: Scenario 15 Dashboard: Visualizing the Global Optimum across all financial, technical, and governance constraints

Analysis of Divergence

The most significant finding is that the heuristic baseline failed to function. By exceeding the CAPEX limit by $2,000, it produced an infeasible solution.

Why the Heuristic Failed: The Sequential Allocation Trap

The failure resulted from Sequential Siloed Execution. The heuristic calculated requirements for Azure and AWS independently. It prioritized the mandatory Azure policy first ($C_{Azure} \geq 175$), consuming most of the capital budget. When it attempted to satisfy the AWS demand, the remaining liquidity was insufficient. Because it optimizes sequentially, the baseline was unable to perform the necessary arbitrage.

The Multi-Cloud Advantage of the LP Solver

The LP solver succeeded by performing "Global Resource Arbitration". Treating the budget as a single shared pool, it recognized that the high capital cost of the Azure allocation required a compensatory reduction elsewhere. It balanced this by aggressively leveraging the Spot

Risk budget on AWS, increasing Spot utilization from 45 to 72 vCPUs. It effectively traded technical risk (Spot usage) for financial liquidity (CAPEX preservation), a complex arbitrage impossible for linear, rule-based scripts.

Chapter 5

Summary and Conclusions

5.1 Synthesis of Research Findings

This thesis addressed the critical engineering challenge of cloud cost optimization. It focused on environments that have surpassed the cognitive capacity of manual management. As established in Chapter 1, the modern cloud landscape is characterized by Rapid Elasticity [17]. It contains an exploding catalog of options, currently exceeding 750 distinct instance types on platforms like AWS. This complexity exceeds the ability of human administrators to govern effectively. The result is a structural Efficiency Gap where standard heuristic tools fail to account for the dynamic variables of multi-cloud pricing.

The research hypothesis posited that standard commercial tools only optimize locally. They aim for immediate cost reduction but are mathematically incapable of solving the conflicting constraints of enterprise infrastructure. To test this, Chapter 3 detailed the development of a deterministic optimization engine. This engine used the Python `PuLP` library and the CBC solver [19]. It was validated against an experimental matrix of 15 operational scenarios. These scenarios simulated real-world friction between engineering performance floors, financial budget caps (CAPEX), and risk tolerances.

The engine demonstrated high computational scalability. It resolved the complex Scenario 15 problem in just 0.031 seconds. The results obtained in Chapter 4 confirm the hypothesis. They demonstrate that the LP approach consistently outperforms heuristic baselines by treating infrastructure as a unified, multi-dimensional solution space rather than a series of isolated decisions [9].

Key findings include:

- **Storage Lifecycle Arbitrage (Scenarios 5 & 13):** The LP solver exposed the limitations of static safety rules used by heuristics. In Scenario 5, the model achieved a 59.56% efficiency gain by dynamically tiering data across Hot, Cool, and Archive storage. In Scenario 13, it identified that a substantial upfront migration investment could yield a 62.7% TCO reduction. This is a long-term ROI that greedy algorithms missed entirely due to their aversion to immediate transition fees. This confirms that saturating CAPEX can result in greater OPEX optimization. This strategy is unachievable for heuristic tools because they cannot predict beyond local time horizons.
- **Constraint Saturation (Scenario 6):** Heuristic tools often leave capital budgets utilized at 0% due to risk aversion. In contrast, the LP model successfully saturated the constraints. It utilized 97.8% ($24,450) of the available CAPEX budget to secure deeper Reserved Instance discounts. This proves that true optimization requires operating at the mathematical edge of financial limits rather than staying safely in the middle.
- **Multi-Objective Governance (Scenario 15):** The final Master Problem applied conflicting constraints for CAPEX ($45k), Spot Risk (75 vCPU), and Azure Policy compliance simultaneously. The LP engine achieved a 13.05% global efficiency gain. It saved $72,138 compared to the heuristic baseline, which failed to maintain budget compliance. This clarifies the capability to find optimizations even within strict boundaries.

These findings validate that combining mathematical modeling with business intelligence visualization effectively bridges the efficiency gap identified by Storment & Fuller [22]. This moves FinOps from reactive reporting to proactive, algorithmic governance. Simple reactive behavior is no longer sufficient; passive strategies do not result in memory optimizations or sustainable enterprise growth.

5.2 The Deterministic Advantage: Linear Programming vs. Heuristics

The most valuable finding in this research was the confirmation of LP's transparency and efficiency. As Dantzig formulated, the Simplex algorithm searches the entire region of possible solution vectors to identify the absolute global optimum [9].

Based on the results from Scenarios 1 through 15, three key advantages of the LP model over the "Best Guess" approach were identified:

1. **Mathematical Optimality vs. Tunnel Vision:** Standard heuristics often focus on single metrics, such as just CPU or just Price. They typically rely on averages across all metrics. The proposed model solves multi-dimensional problems simultaneously. This was clearly visible in Scenario 4, where the solver right-sized for both CPU and RAM. It was also evident in Scenario 5, where it tiered storage across Hot, Cool, and Archive levels to maximize savings.

2. **Saturation of Constraints:** Heuristic tools often leave resources unoptimized due to the fear of hitting limits. They avoid using CAPEX to its full extent because they cannot estimate the potential long-term efficiency gains. The LP solver maximizes utility by saturating these constraints exactly. For example, in Scenario 6, the model utilized $24,450 of a $25,000 budget (97.8%), whereas the heuristic spent $0 to prevent potential risks. Similarly, in Scenario 3, the solver utilized 99% of the available risk budget to maximize Spot instance usage. This proves that true efficiency requires pushing operations to the safety boundary defined by engineering requirements.

3. **Minimization of Waste:** The deterministic approach minimizes unused elasticity. In the experiments, the solver reduced provisioned capacity from 36 to 34 vCPUs while still meeting demand. This prevented the overspending seen in the heuristic baseline.

5.3 Operational Trade-offs and Implementation Costs

While the mathematical results are strong, implementing this solution in a real engineering environment involves significant trade-offs. The greatest difficulty is not the math itself, but the implementation of the environment, specifically regarding data collection and computational requirements.

Computational Overhead and Speed

The engineering effort must be assessed relative to the financial return. In simple, highly constrained environments (Scenarios 12 & 14), the LP model yielded marginal gains of 1.7% to 4%. However, the system's primary ROI is realized in complex, multi-variable scenarios. Specifically, Storage Lifecycle Arbitrage (Scenario 13) and Storage Tiering (Scenario 5) demonstrated TCO reductions of 62.7% and 59.56%, respectively. These high-impact outcomes justify the initial development overhead; a single successful optimization in storage architecture effectively amortizes the engineering cost of the entire platform.

Complexity of Implementation

Unlike SaaS tools that are "plug and play," this approach requires a specialized skill set. The system's effectiveness is strictly bound by the quality of input data. As VanderLans notes, data integrity is paramount [23]. If the telemetry is flawed ("garbage in"), the solver will output a result that is mathematically perfect but operationally useless ("garbage out").

Furthermore, limitations were observed in highly constrained environments. In Scenarios 12 and 14, strict policies shrank the feasible solution space (e.g., "No Spot Instances allowed" or "Azure Only"). In these cases, the LP model yielded minimal gains of only 3-4% over the baseline. This confirms that the model is realistic; it does not fabricate savings where none exist. It also suggests that for simple, rigid environments, the "heuristic tax" might be acceptable compared to the high effort of implementing a custom LP engine.

5.4 Final Conclusions

This thesis successfully validated the application of Linear Programming as a viable methodology for cloud cost optimization. By replacing reactive heuristics with proactive algorithmic logic, the system transformed simulated usage data into an actionable financial strategy.

The integration of Python-based solvers with Microsoft Power BI provided the transparency needed to make complex mathematical findings understandable to stakeholders [21]. However, this is not a universal solution for every company. It is best suited for complex, enterprise-scale environments where the 10-30% efficiency gap justifies the investment in data engineering. Ultimately, it is concluded that the future of FinOps lies in the automated, deterministic governance of infrastructure through linear optimization, moving beyond simple dashboards to true mathematical control.

Chapter 6

Project Evolution and Future Development Paths

6.1 Evolution of the Thesis Concept

The original objective of the research was to input historical cost reports to identify static instances of waste, such as unattached storage volumes or idle compute nodes. However, early literature review and technical prototyping revealed that this approach merely replicated the functionality of existing "Level 1" heuristic tools. It offered limited academic novelty or engineering depth.

Consequently, the research scope evolved from simple accounting to the development of a comparative optimization engine. The focus shifted toward showcasing the existing "Efficiency Gap" between standard human decision-making (heuristics) and the mathematical determinism of Linear Programming. Infrastructure provisioning was treated not as an administrative task, but as a high-dimensional vector-optimization problem. The project transitioned into an engineering study of algorithmic efficacy in the face of the growing complexity of cloud markets [6].

6.2 Research Limitations and Data Acquisition Challenges

A significant obstacle in the engine's development was data acquisition. To operate on real-world inputs, requests for anonymized logs were submitted to major enterprises, including

Nordea, Nokia, and TE Connectivity. However, strict privacy policies and Non-Disclosure Agreements (NDAs) prevented these organizations from sharing proprietary billing data.

To ensure the engine could still model real-world economics, random synthetic data was deemed insufficient. Randomness would result in the loss of specific economic relationships between instance types. Therefore, accurate pricing information was retrieved directly from service providers via the AWS and Azure APIs. This resulted in a "hybrid" model: synthetic demand patterns mapped against real-world financial constraints.

The shift to synthesized data improved the analytical comparison. By controlling the input vectors, the risk of flawed telemetry was avoided. The research established a "Control Group"—the Heuristic Baseline—against which the Linear Programming model's optimized TCO could be compared with mathematical precision. This approach ensured reproducibility and allowed the "Efficiency Gap" to be quantified without interference from data quality issues inherent in raw corporate logs.

However, future development requires transitioning beyond the controlled environment. While synthetic data validated the mathematical core, the system must be stress-tested against the "noise" of real-world corporate telemetry. Transitioning to raw, unstructured logs in a production environment is essential to evaluate how the solver handles the complexities of daily cloud operations.

6.3 Future Directions: From Script to Platform

The current implementation validates the core mathematical hypothesis. However, it operates as a local engineering tool within a Google Colab environment and requires Python proficiency. To bridge the gap between thesis validation and industrial application, the model requires further development in several key areas.

6.3.1 Transition to SaaS Architecture

The current engine relies on local execution, limiting accessibility to technical personnel. A necessary step for project evolution is the migration of the optimization engine to a web-based Software-as-a-Service (SaaS) platform. Future work would focus on wrapping the `PuLP` solver logic within a backend framework capable of handling remote computation requests.

This implementation would separate the optimization logic from the execution environment. Finance and Engineering teams could interact with the model via a RESTful API. Users would upload standard CUR files via a frontend interface, and the engine would return optimization

vectors. This democratization of access aligns with the needs of FinOps teams, who require tools that abstract mathematical complexity behind intuitive user interfaces [22].

6.3.2 Integration of Predictive Machine Learning

Machine learning is becoming crucial for enterprise infrastructure. Assuming the availability of structured, noise-free input data, this engine serves as a foundation for advanced AI integration. Future improvements could include predictive vector optimization, moving beyond optimizing "current" spend to forecasting "future" requirements.

ML-Enhanced Forecasting

Implementing time-series forecasting models, such as ARIMA or LSTM, enables a shift from reactive to proactive provisioning. Zhang et al. (2021) emphasize that intelligent financial operations in multi-cloud environments demand such predictive analytics combined with deterministic control [29].

6.3.3 Stress-Testing in Non-Ideal Telemetry Environments

This research validated the mathematical core of the engine under ideal conditions. However, real-world cloud environments are rarely clean. Operational telemetry often suffers from "noise," including inconsistent tagging, delayed billing data, or broken usage records. Therefore, a critical next step is moving from validation to Resilience Testing.

Future development should focus on "Sensitivity Analysis" using controlled error injection. Instead of ingesting raw production logs immediately, the data generation pipeline should be modified to introduce random anomalies. Programmatically injecting errors, such as null values for CPU utilization or unmapped storage tags, allows for the measurement of failure boundaries.

This step is necessary to determine the model's tolerance. It identifies how much data degradation the Linear Programming model can withstand before the calculated "Global Optimum" becomes unreliable. Hardening error-handling logic is a necessary prerequisite before connecting the engine to raw, real-world data pipelines.

6.3.4 High-Dimensional Scalability and Latency Benchmarking

Scenario 15 validated the engine against a core set of cost and performance variables. However, industrial environments impose a higher number of restrictions. A necessary evo-

lution involves benchmarking the solver against complex policy logic. The model must be stress-tested with increasing simultaneous constraints, ranging from standard budget pools to overlapping regulatory requirements like GDPR residency and PCI-DSS compliance.

The objective is to map the solver's execution time against this increasing complexity. The upper limit of constraints must be determined to ensure the system maintains the sub-second response times (0.031s) achieved in the current baseline.

Achieving this stability under high-dimensional load would confirm the platform's viability as a commercial-grade FinOps solution, completing the transition from academic prototype to enterprise product.

Bibliography

[1] Amazon Web Services. Cost optimization pillar - aws well-architected framework, 2024. https://docs.aws.amazon.com/wellarchitected/latest/cost-optimization-pillar/welcome.html [Dostęp: 15.11.2025]. [page 14]

[2] Amazon Web Services. Amazon ec2 instance types. https://aws.amazon.com/ec2/instance-types/, 2025. [Dostęp: 15.11.2025]. [page 8, 13]

[3] Michael Armbrust, Armando Fox, Rean Griffith, Anthony D Joseph, Randy Katz, Andy Konwinski, Gunho Lee, David Patterson, Ariel Rabkin, and Ion Stoica. A view of cloud computing. *Communications of the ACM*, 53(4):50–58, 2010. [page 8, 14, 21, 50, 51, 53]

[4] Anton Beloglazov, Jemal Abawajy, and Rajkumar Buyya. Energy-aware resource allocation heuristics for efficient management of data centers for cloud computing. *Journal of Parallel and Distributed Computing*, 72(10):1272–1283, 2012. [page 9, 11, 17, 21, 39]

[5] Dimitris Bertsimas and John N. Tsitsiklis. *Introduction to Linear Optimization*. Athena Scientific, 1997. Foundational academic textbook on optimization theory. [page -]

[6] Rajkumar Buyya, Chee Shin Yeo, Srikumar Venugopal, James Broberg, and Ivona Brandic. Cloud computing and emerging it platforms: Vision, hype, and reality. *Future Generation Computer Systems*, 25(6):599–616, 2009. [page 8, 51, 60]

[7] S. Chaisiri, B. S. Lee, and D. Niyato. Optimization of resource provisioning cost in cloud computing. *IEEE Transactions on Services Computing*, 5(2):164–177, 2012. [page 9, 16, 22, 36]

[8] Niangjun Chen, Joshua Comden, Zhenhua Liu, et al. Online optimization in cloud resource provisioning: Predictions, regrets, and algorithms. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 3(1), 2019. [page 9, 17, 39]

[9] George B. Dantzig. *Linear Programming and Extensions*. Princeton University Press, 1998. [page 9, 12, 16, 18, 36, 43, 56, 57]

[10] Greg Deckler and Brett Powell. *Mastering Microsoft Power BI: Expert techniques to create interactive insights for effective data analytics and business intelligence*. Packt Publishing, 2nd edition, 2022. Comprehensive guide for advanced Power BI implementation. [page -]

[11] Saurabh Deochake. Cloud cost optimization: A comprehensive review, 2023. [Dostęp: 15.11.2025]. [page 15]

[12] Alberto Ferrari and Marco Russo. *The Definitive Guide to DAX: Business intelligence for Microsoft Power BI, SQL Server Analysis Services, and Excel*. Microsoft Press, 2019. [page 19]

[13] Stephen Few. *Information Dashboard Design: The Effective Visual Communication of Data*. O'Reilly, 2006. [page -]

[14] Aurélien Géron. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media, 3rd edition, 2022. Essential for the "Future Directions: Predictive ML" chapter. [page -]

[15] Yves Hilpisch. *Python for Finance: Mastering Data-Driven Finance*. O'Reilly Media, 2nd edition, 2018. Standard reference for modeling financial constraints in Python. [page -]

[16] Wes McKinney. *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter*. O'Reilly Media, 3rd edition, 2022. [page -]

[17] Peter Mell and Timothy Grance. The nist definition of cloud computing. Technical Report SP 800-145, National Institute of Standards and Technology, 2011. [Dostęp: 15.11.2025]. [page 8, 10, 14, 56]

[18] Microsoft. *Get started with Power BI Desktop*. Microsoft, 2025. `https://learn.microsoft.com/en-us/power-bi/fundamentals/desktop-getting-started` [Dostęp: 15.11.2025]. [page -]

[19] Stuart Mitchell, Michael O'Sullivan, and Iain Dunning. Pulp: A linear programming modeler for python. *Optimization Online*, 65, 2011. [page 12, 18, 22, 35, 56]

[20] PuLP Development Team. *PuLP: A Linear Programming API for Python*. COIN-OR Foundation, 2025. `https://coin-or.github.io/pulp/` [Dostęp: 15.11.2025]. [page -]

[21] Ramesh Sharda, Dursun Delen, and Efraim Turban. *Business Intelligence, Analytics, and Data Science: A Managerial Perspective*. Pearson, 4th edition, 2020. [page 19, 59]

[22] J. R. Storment and Mike Fuller. *Cloud FinOps: Collaborative, Real-Time Cloud Financial Management*. O'Reilly Media, 2nd edition, 2023. [page 10, 11, 13, 14, 35, 45, 57, 62]

[23] Rick VanderLans. *Unified Data Management*. Technics Publications, 2020. [page 14, 20, 59]

[24] S. Varase, S. Wagh, and P. Kadam. Cloud computing resource scheduling and cost optimization: A survey. *International Journal of Advanced Computer Science and Applications*, 12(4):468–475, 2021. [page -]

[25] Pauli Virtanen et al. Scipy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, 17:261–272, 2020. [page 22]

[26] H. Paul Williams. *Model Building in Mathematical Programming*. John Wiley & Sons, 5th edition, 2013. The classic text on translating business problems into linear equations. [page -]

[27] Laurence A. Wolsey. *Integer Programming*. John Wiley & Sons, 2nd edition, 2020. [page -]

[28] Wydział Telekomunikacji, Informatyki i Elektrotechniki. Regulamin dyplomowania. `http://wtie.utp.edu.pl/dla-dyplomantow/`. [Dostęp: 30.06.2023]. [page -]

[29] Y. Zhang, K. Wang, and H. Liu. Intelligent cloud operations management: A survey of emerging trends. *IEEE Access*, 9:12345–12356, 2021. [Dostęp: 15.11.2025]. [page 12, 15, 38, 47, 62]

List of Figures

1.1 Comparison of AWS EC2 Pricing Models: On-Demand, Reserved, and Spot Instances 9
1.2 The FinOps Lifecycle: Inform, Optimize, and Operate 10

2.1 **The FinOps Lifecycle:** The crucial phases of FinOps, which are Inform, Optimize, Operate 15
2.2 **Possible Solution Area Representation:** The solution space that is limited by technical constraints (blue) and budgetary limits (red). The Simplex algorithm is going through all the vertices and vectors in a search for the Global Optimum . . 16
2.3 **Local vs. Global Optimization:** Heuristic methods often settle for a local optimum (A), while LP solvers evaluate the full constraint set to identify the global optimum (B) 18

3.1 Cloud-Hosted Computational Environment (Google Colab) utilizing the IPython kernel 24
3.2 Serialization of optimization outputs into CSV files for visualization ingestion . . . 24
3.3 Selection of specific scenario files for import into the visualization layer 25
3.4 Initialization of the data import process via the Power BI data input interface . . . 25
3.5 Schema validation and data type transformation during the import preview phase . 26
3.6 The consolidated data model containing independent tables for all 15 scenarios . . 27
3.7 Console output for Scenario 1 demonstrating sub-millisecond solver performance (0.006s) 27
3.8 Console output for Scenario 15 verifying efficiency in multi-constraint environments (0.031s) 28
3.9 The limitation of Native Tools (Provider Silos) vs. the Unified Optimization of the LP Engine 28

4.1 Scenario 1 Dashboard: Visualizing the optimal saturation of Reserved Instances against baseline demand . . . 36
4.2 Scenario 2 Dashboard: The efficiency frontier where the LP solver saturates the budget constraint . . . 38
4.3 Scenario 3 Dashboard: Visualizing the cost-to-risk frontier and the saturation of the risk budget . . . 39
4.4 Scenario 4 Dashboard: Comparative analysis of provisioned capacity versus utilized demand vectors . . . 41
4.5 Scenario 5 Dashboard: Storage Tiering Arbitrage . . . 42
4.6 Scenario 9 Dashboard: The trade-off between commitment liability and discount yield . . . 44
4.7 Scenario 7 Dashboard: The utilization of the "Risk Budget" to drive down unit costs 46
4.8 Scenario 11 Dashboard: Visualizing the Multi-Cloud Budget Allocation and arbitrage between AWS and Azure capacity . . . 49
4.9 Scenario 13 Dashboard: Breakeven analysis of Storage Migration, highlighting the ROI of upfront transition costs . . . 52
4.10 Scenario 15 Dashboard: Visualizing the Global Optimum across all financial, technical, and governance constraints . . . 54

List of Tables

4.1 Scenario 1: Simulation Parameters 36
4.2 Comparative Performance: Heuristic vs. LP Solver (Scenario 1) 36
4.3 Scenario 2: Budget and Performance Vectors 37
4.4 Comparative Performance: Heuristic vs. LP Solver (Scenario 2 - $10k Budget) . . 37
4.5 Scenario 3: Risk Parameters 39
4.6 Comparative Performance: Heuristic vs. LP Solver (Scenario 3) 39
4.7 Scenario 4: Vector Matching Parameters 40
4.8 Comparative Performance: Heuristic vs. LP Solver (Scenario 4) 40
4.9 Scenario 5: Storage Tiering Costs 42
4.10 Comparative Performance: Heuristic vs. LP Solver (Scenario 5) 42
4.11 Scenario 6: Financial Constraints and Performance 43
4.12 Scenario 9: Liability vs. Discount Optimization 44
4.13 Scenario 7: Risk Budgeting and Spot Utilization 45
4.14 Scenario 8: Policy Enforcement Cost Impact 46
4.15 Scenario 10: Storage Tiering Optimization 47
4.16 Scenario 11: Multi-Cloud Budgetary Performance 48
4.17 Scenario 11: Financial Structure Breakdown (CAPEX vs. OPEX) 48
4.18 Scenario 12: Cost of Risk Aversion & Performance Guarantees 50
4.19 Scenario 14: Corporate Policy Compliance Cost 51
4.20 Scenario 14: Analysis of Compliance Precision 51
4.21 Scenario 13: Storage Lifecycle Economics 52
4.22 Scenario 13: Financial Structure & Return on Investment (ROI) 52
4.23 Scenario 15: Master Problem Results 53
4.24 Scenario 15: Financial Feasibility Analysis (CAPEX/OPEX) 54

List of Attachments

The following digital components are provided as supplementary material to this thesis:

1. **Folder: CSV Results**
   Contains the raw data and output files generated for all tested scenarios.
2. **PowerBI Dashboard**
   The `.pbix` file containing the interactive data visualizations and comparative analysis.
3. **Folder: Python Source Code**
   The complete implementation of the algorithms, including specific scripts for each scenario and data processing modules.